ETAS RTA-BSW v12.7.0

RTA-BSW MISRA Compliance Statement

www.etas.com

Copyright

The data in this document may not be altered or amended without special notification from ETAS GmbH. ETAS GmbH undertakes no further obligation in relation to this document. The software described in it can only be used if the customer is in possession of a general license agreement or single license. Using and copying is only allowed in concurrence with the specifications stipulated in the contract.

Under no circumstances may any part of this document be copied, reproduced, transmitted, stored in a retrieval system or translated into another language without the express written permission of ETAS GmbH.

© Copyright 2025 ETAS GmbH, Stuttgart

The names and designations used in this document are trademarks or brands belonging to the respective owners.

ETAS RTA-BSW 12.7.0 - RTA-BSW MISRA Compliance Statement R01 EN – 05.2025

Contents

1 RTA-BSW MISRA 2012 Compliance Statement 4

1.1 Introduction 4

1.2 AUTOSAR mandated violations 4

1.3 MISRA Directive 1.1 and rules 5.1 to 5.5 4

1.4 MISRA Directive 4.10 and Rule 20.1 4

2 RTA-BSW Stacks 5

2.1 Common 5

2.2 RTA-BASE 8

2.3 RTA-CAN 10

2.4 RTA-COM 12

2.5 RTA-DIAG 21

2.6 RTA-ETH 31

2.7 RTA-ETHSEC 45

2.8 RTA-FRAY 47

2.9 RTA-HWD(Can) 51

2.10 RTA-HWD(Eth) 51

2.11 RTA-HWD(Fray) 52

2.12 RTA-HWD(Lin) 53

2.13 RTA-J1939 53

2.14 RTA-LIN 56

2.15 RTA-MEM 57

2.16 RTA-RipsPlugins 62

2.17 RTA-SAFE 68

2.18 RTA-SEC 70

2.19 RTA-SWCL 82

2.20 RTA-XCP 82

3 Contacting ETAS 88

3.1 Technical Support 88

3.2 ETAS HQ 88

3.3 ETAS Local Sales and Support Offices 88

1 RTA-BSW MISRA 2012 Compliance Statement

1.1 Introduction

RTA-BSW consists of a large number of modules grouped into stacks, which are documented in the RTA-BSW reference guides (see `*_Reference_Guide_EN.pdf` in the RTA-BSW documentation directory). An ECU project which uses RTA-BSW modules will configure and generate an ECU specific version of the module using ETAS ISOLAR-AB.

This MISRA compliance statement follows the reference guide structure, with individual sections for each stack containing a subsection for each module within the stack. Each module subsection lists the MISRA directives and rules which may be violated by the generated module. The code generated for each configured component contains structured comments of the form `/* MR12 RULE 1.2 VIOLATION: justification */` where `justification` explains the reason for the specific violation at that point in the generated code. The lists in this section are provided as an overview of the violations that should be expected.

When integrating RTA-BSW into a MISRA compliant project, the integrator should review the generated code for commented rule violations, and check that each violation is acceptable for the project.

1.2 AUTOSAR mandated violations

Some aspects of the AUTOSAR design require MISRA violations and therefore impact many RTA-BSW modules. The most common are shown here.

1.3 MISRA Directive 1.1 and rules 5.1 to 5.5

C90 only requires that the first 6 characters of an identifier are significant. However the AUTOSAR naming scheme for both data and functions assumes that the compiler treats the whole identifier as significant. This violates MISRA 2012 DIR 1.1 and rules 5.1 to 5.5.

1.4 MISRA Directive 4.10 and Rule 20.1

The memory mapping mechanism used by the generated RTE and described by the AUTOSAR standard in `AUTOSAR_SWS_MemoryMapping.pdf` violates MISRA 2012 Dir 4.10, Rule 20.1.

2 RTA-BSW Stacks

2.1 Common

2.1.1 Bfx

No violations

2.1.2 Compiler

- RULE 20.7: It is not possible to use brackets here as per AUTOSAR concept.

2.1.3 ComStack

No violations

2.1.4 Crc

- Rule 14.2 Violation: Loop control variable in this 'for' statement, index, is modified in the body of the loop. Does not lead to any functional issue
- Rule 15.7 Violation: No concluding 'else' exists in this 'if'-'else'-'if' statement. Does not lead to any functional issue

2.1.5 EcuC

No violations

2.1.6 Efx

- DIR 4.1: Code is checked that expression is correct, see lines above.
- RULE 15.4: First break was inserted to improve runtime and cannot be removed Second break is neccesary and is part of the sort algorithm.
- RULE 19.2: A union is used, without usage of union speed will decrease.
- RULE 19.2: Union is used for faster computation, without usage of union code size will increase.
- RULE 8.13: The interface is defined with present prototye within AUTOSAR specification
- RULE 8.13: The interface is specified in AUTOSAR without a const.
- RULE 8.13: The object is modified within the inline assembler code.

2.1.7 Ifl

- DIR 1.1: Float is used in union to extract bit pattern.
- RULE 11.5,DIR 1.1 : 64bit copy action of 8byte aligned structure is required for data consistency.
- RULE 14.1:Floating point comparison is required in order ensure the correct behavior of the function

- RULE 19.2: Union is used for bit-field extraction of floating point variable which is used for certain operations on floating point variable.
- RULE 19.2:Union is used for bit-field extraction of floating point variable which is used for certain operations on floating point variable.

2.1.8 Ifx

- RULE 11.5,DIR 1.1 : 32bit copy action of 32bit aligned structure is required for data consistency.

2.1.9 Mfl

- DIR 1.1: Comparision is needed. This check could be replaced if a service is available
- DIR 1.1: Float is used in union to extract bit pattern.
- RULE 10.4: Calculation is done correctly.
- RULE 14.1:Floating point comparison is required in order ensure the correct behavior of the function
- RULE 8.13: The interface is defined with present prototye within AUTOSAR specification

2.1.10 Mfx

- RULE 19.2: Union is used for faster computation, without usage of union code size will increase.

2.1.11 Platform

- DIR 1.1: the type _Bool is mapped here to the AUTOSAR type boolean to prevent the direct use of the type _Bool

2.1.12 rba_ArxmlGen

No violations

2.1.13 rba_BswSrv

- DIR 4.1, RULE 1.3, 11.4: The warning is not critical and non-removable since casting between integers and pointers is required to interpret the address. Function below is needed because C99 _Alignof() is not available in our enviroment There is no real NULL-pointer dereference.
- RULE 11.3: The warning is non-removable since casting is required to manipulate the data as uint8/uint16 type. The warning is uncritical since the data size is reduced and therefore the alignment is even better than required.
- RULE 11.3: The warning is non-removable since casting is required to manipulate the data as uint8/uint16/uint32 type. The warning is uncritical since the

data size is reduced and therefore the alignment is even better than required.

- RULE 11.4,11.6: The warning is non-removable since casting is required to manipulate the data as uint8/uint16/uint32 type. The warning is uncritical since the address range and proper alignment is ensured by the calculation done befobamfre in integer representation.
- RULE 11.4,11.6: The warning is non-removable since casting is required to manipulate the data as uint8/uint16/uint32 type. The warning is uncritical since the address range and proper alignment is ensured by the calculation done before in integer representation.
- RULE 11.4,11.6: The warning is uncritical and non-removable since casting is required to manipulate memory address as uint32.
- RULE 11.4: The Warning is uncritical and non-removable since casting between integers and pointers is required to interpret the address
- RULE 11.4: The Warning is uncritical and non-removable since casting between integers and pointers is required to interpret the address.
- RULE 11.5, 11.6: The Warning is uncritical and non-removable since conversion from a pointer to object into a pointer to void is required to access buffer
- RULE 11.5: The Warning is uncritical and non-removable since conversion from a pointer to object into a pointer to void is required to interpret the address.
- RULE 11.5: The warning is non-removable since pointer to void is required to have same syntax as C90 interface and casting is required to manipulate the data. The warning is uncritical since the address range and proper alignment is ensured by the calculation done before in integer representation.
- RULE 13.3: The warning is not critical and non-removable since value using from postfix increment is required for code optimization
- RULE 13.3:The warning is not critical and non-removable since value using from postfix increment is required for code optimization
- RULE 18.7: The warning is not critical and non-removable since Structure is used to access variable sized objects with “overlay pointers” This is allowed with C99 (which we use).
- RULE 21.2: In order to support QAC_2777 and QAC_2776 warning of MISRA 2012 Mandatory, rba_bswsrv_memcopy function is mapped to memcpy by Cchecker Tool. This mapping leads to generation of warnings QAC_4603 and QAC_4604, which are false positives and could not be removed.
- RULE 8.6: INLINE functions will be defined in Header
- RULE 9.1: variable is set by inline assembly but the checker does not understand this
- Rule 11.5: The Warning is uncritical and non-removable since conversion from a pointer to object into a pointer to void is required to interpret the address.

- Rule 18.3: The Warning is uncritical and non-removable since comparisons between pointers is required to get the address of the next message
- Rule 18.3: The Warning is uncritical and non-removable since comparisons between pointers is required to set the next address to be pointed
- Rule 18.3: The Warning is uncritical and non-removable since comparisons between pointers is required to set the next address to be pointed.

2.1.14 rba_DiagLib

- DIR 1.1: Function rba_BswSrv_MemCompare converts argument 1 and 2 to pointer to void, function has no side effects
- DIR 1.1: Function rba_BswSrv_MemCopy converts argument 1 and 2 to pointer to void, function has no side effects
- DIR 1.1: Function rba_BswSrv_MemSet converts argument 1 to pointer to void, function has no side effects
- RULE 11.9: The first row is a invalid index and will never be used
- RULE 2.7: the parameter 'indx' is not used because the retrun value is the same for all indices due to the config, it is kept because the code calling the function is not config base but static

2.1.15 Standard

No violations

2.2 RTA-BASE

2.2.1 BswM (BSW Mode Manager)

- DIR 1.1: BswM_Prv_Evaluate_Action expects a void pointer to actionlistitem which is then typecast back to the corresponding action type
- DIR 1.1: Identifier name depends on the configured pre-defined variants hence the first 63 significant characters of one macro may match macro name generated for another config set.
- RULE 11.5: Since only the configuration will determine whether the action list item is another action list or not, it is assigned to void pointer.It is low prio because it is typecast to the corresponding action list or action item type after first checking for the same in this function.
- RULE 11.5: The original configuration is of the specific action type, which is then typecast to void and passed as void pointer by BswM_Prv_Evaluate_Action and then typecast back to the corresponding action type here.
- RULE 12.3, 13.4: MISRA Rule 12.3 Inline optimization leads to use of comma operator outside for statement. MISRA Rule 13.4 Inline optimization leads to use of assignment operator.
- RULE 17.2: If the action item is a reference to another action list then this function needs to be called recursively.Currently the action list structure

may contain only up to 7 hierarchic levels so it will not block code execution.

- RULE 5.5:identifier name depends on the configured pre-defined variants hence the first 31 significant characters of one macro may match macro name generated for another config set

2.2.2 BswM_PB_Variant_Only

No violations

2.2.3 BswM_PreCompile_and_PB_Variant

No violations

2.2.4 Det (Default Error Tracer)

- RULE 17.2: The Dlt_DetForwardErrorTrace processes Det errors to DLT other than DLT module itself to avoid recursion ensured by module ID check
- RULE 8.5: Duplicate of Det_ReportTransientFault to make it also available in the GeneralDiagnosticInfo RTE interface
- RULE 8.5: Duplicate of Det_ReportError to make it also available in the GeneralDiagnosticInfo RTE interface
- RULE 8.5: Duplicate of Det_ReportRuntimeError to make it also available in the GeneralDiagnosticInfo RTE interface

2.2.5 EcuM (ECU State Manager)

- RULE 8.13: The pointer parameter will only be modified for an EcuM configuration with NvM enabled
- RULE 8.4: The declaration of GetCoreID is expected from Os

2.2.6 StbM (Synchronized Time-Base Manager)

- DIR 1.1: A 'pointer-to-object' conversion is necessary in this case
- RULE 10.3: l_Deltatick_uo is of type TickType. Hence the conversion is necessary in this case
- RULE 10.3: l_tick is of type TickType. Hence the conversion is necessary in this case
- RULE 10.4,10.8: nanoseconds to ticks conversion is done depending on the frequency
- RULE 10.4,10.8: ticks to nanoseconds conversion is done depending on the frequency
- RULE 10.8: RateDeviation can be positive or negative hence this conversion is required
- RULE 16.4: default case has to be empty as no action should be taken as per AR requirement

2.2.7 Tm

No violations

2.3 RTA-CAN

2.3.1 CanIf (CAN Interface)

- RULE 1.3,13.2 : Warning is “Not Critical”.Multiple volatile reads.No issue in code.
- RULE 10.3:The Warning is “Not Critical”. The left operand of shift operator is lower in size compared to right operand, as the left operand is only used to store three bits for which unit8 is sufficient
- RULE 10.3:The Warning is “Not Critical”. The left operand of shift operator is lower in size compared to right operand, as the left operand is only used to store two bits for which unit8 is sufficient
- RULE 10.3:The Warning is “Not Critical”. The operand SduLength is reduced in size to uint8. The length of pdu will be maximum 64 which is covered in uint8. Thus there is no issue in reducing the size.
- RULE 10.4, 10.1:The Warning is “Not Critical”. The left hand operand is reduced in size compared to right operand, as the left operand is only used to store two bits for which unit8 is sufficient
- RULE 10.8, 10.5:Warning is “Not Critical”.Explicit conversion of composite expression
- RULE 10.8, 10.5:Warning is “Not Critical”.The left operand of shift operator is lower in size compared to right operand, as the left operand is only used to store two bits for which unit8 is sufficient
- RULE 11.3, 11.8: Warning is “Not Critical”. The pointer cast is required her.
- RULE 11.3: Warning is “Not Critical”. The pointer cast is required here for misaligned case. The pointer cast is required here
- RULE 11.4: Warning is “Not Critical”.Cast from pointer to object to integral type is OK.
- RULE 11.5 : ‘pointer-to-void’ is being cast to a ‘pointer-to-object’ type and this conversion will not result in any misalignment.
- RULE 11.5 : Warning is “Not Critical”.’pointer-to-void’ is being cast to a ‘pointer-to-object’ type and this conversion will not result in any misalignment.
- RULE 11.5, 11.8 : ‘pointer-to-void’ is being cast to a ‘pointer-to-object’ type and this conversion will not result in any misalignment.
- RULE 11.5,11.8 10.4 : ‘pointer-to-void’ is being cast to a ‘pointer-to-object’ type and this conversion will not result in any misalignment.
- RULE 11.5,11.8 : ‘pointer-to-void’ is being cast to a ‘pointer-to-object’ type and this conversion will not result in any misalignment.
- RULE 12.1: Warning is “Not Critical”.Verified through testing that rules of

precedence and associativity are not affecting the functionality

- RULE 13.5:The Warning is “Not Critical”. Brackets are used correctly to prevent any side effects of binary operator precedence
- RULE 14.3,: Warning is “Not Critical”. DET condition is intentionally made as True to report DET error as the transmit CAN frame is requested when Dlc to be transmitted is greater then configured when truncation is disabled
- RULE 14.3: Warning is “Not Critical” as the controlling expression is already checked before entering this condition
- RULE 15.4:The Warning is “Not Critical”. Multiple break points are used for optimal operation of filtering feature
- RULE 20.7,: This Warning is “Not Critical”. The parameter SCHM_EXIT_FUNC is used to function call.
- RULE 8.13: params_pst variable is without const.

2.3.2 CanNm (CAN Network Management)

- RULE 11.5: Pointer is intialized for Post Build support
- RULE 8.13: A pointer parameter in a function prototype should be declared as pointer to const if the pointer is not used to modify the addressed object. But AUTOSAR SWS specification signature is as below and the PduInfoPtr will be modified if the user data is enabled, Hence suppressed

2.3.3 CanNm_PB_Variant_Only

No violations

2.3.4 CanNm_PreCompile_and_PB_Variant

No violations

2.3.5 CanSM (CAN State Manager)

- RULE 13.5: Warning is “Not Critical”. Calibration functions are being called on the right hand side of the logical && operation to obtain the L1 and L2 timer values. Those functions will return a uint16 value, hence it doesnt have any side effects. Redesign is not recommended as the change would decrease maintainability and increase runtime
- RULE 13.5: Warning is “Not Critical”. CanSM_CheckTxRxNotification() is being called on the right hand side of the logical && operation to check the Tx and Rx status in polling mode. It is ensured that this function doesn’t have any side effects Redesign is not recommended as the change would decrease maintainability and increase runtime

2.3.6 CanTp (CAN Transport Layer)

- RULE 10.4, 14.3: As per ISO this value should be constant when we receive reserved STmin value in FC. MISRA C:2012 Rule- 10.4,14.3

- RULE 10.4: As per ISO this value should be constant when we receive reserved STmin value in FC. MISRA C:2012 Rule-10.4
- RULE 5.6 , 5.7 , 5.9 : the elements have been declared here to define the element type for the generated structures. hence there are no issues
- RULE 5.6,5.7,5.8,5.9 : the elements have been declared here to define the element type and generate the types.h file. hence there are no issues
- RULE 5.7,5.8,5.9 : the elements have been declared here to define the element type and generate the types.h file. hence there are no issues
- RULE 8.4: Declarartion for CanTp_XX_YY_ExternalFdSupportCallback is present indirectly in #define CanTp_FdCallOut, hence this warning is invalid

2.3.7 CanTp_PBL

No violations

2.3.8 CanTp_PBS

No violations

2.3.9 CanTp_PC

No violations

2.3.10 CanTSyn (Time Synchronization over CAN)

- RULE 5.2: Warning is “Not Critical”. The naming convention of Handles is according to AUTOSAR req TPS_ECUC_02108 and hence the length of the name cannot be reduced.

2.4 RTA-COM

2.4.1 Com (Communication)

- DIR 1.1, RULE 13.5: Evaluation(or no evaluation) of the second part of logical-OR shall not impact the system behaviour. Hence suppressed. Explicit cast is provided in line with COM_MEMCOMPARE definition. But the void pointer is always deferenced to a type, based on the pointers alignment.Hence the alignment warning can safely be ignored.
- DIR 1.1, RULE 21.16: 1.signalDataPtr_pcv is defined as void pointer in the AUTOSAR specification, so suppressing warning “Cast from a pointer to object type to a pointer to void”. 2.Memcmp expects valid Type argument to be passed, but here signalDataPtr_pcv is of void type. So, no functional impact.
- DIR 1.1: “Cast from a pointer to object type to a pointer to void”. As below mem-copy is optimized operation to copy data, so, type cast warning is ignored here
- DIR 1.1: “Cast from a pointer to object type to a pointer to void”. rxSrcPtr_pv parameter is always deferenced with correct type, so, there will not be any

impact. Hence, it is suppressed.

- DIR 1.1: “Implicit conversion from a pointer to object type to a pointer to void.” warning can be suppressed as the SignalDataPtr will be dereferenced with the same datatype as used in Com_Prv_InternalSendSignal
- DIR 1.1: “Implicit conversion from a pointer to object type to a pointer to void.” warning can be suppressed as the msgData_pcv will be dereferenced with the same type at destination side
- DIR 1.1: “Implicit conversion from a pointer to object type to a pointer to void.” warning can be suppressed as the msgData_pv will be dereferenced with the same type at destination side
- DIR 1.1: Explicit cast has no impact, as it is de-referenced with same type-qualifier
- DIR 1.1: Explicit cast is provided in line with COM_MEMCOMPARE definition. But the void pointer is always deferenced to a type, based on the pointers alignment. Hence the alignment warning can safely be ignored.
- DIR 1.1: Explicit cast is provided in line with Com_Prv_SendDynSignal() definition.
- DIR 1.1: Explicit cast is provided in line with rba_BswSrv_MemCopy() definition. But the void pointer is always deferenced to a type based on the pointers address alignment. Hence the alignment warning can safely be ignored.
- DIR 1.1: Explicit cast is provided in line with rba_BswSrv_MemCopy() definition. But the void pointer is always deferenced to a type, based on the pointers address alignment. Hence the alignment warning can safely be ignored.
- DIR 1.1: Explicit cast is provided in line with rba_BswSrv_MemCopy() definition. But the void pointer is always deferenced to a type, based on the pointers alignment. Hence the alignment warning can safely be ignored.
- DIR 1.1: pointer structure types are not exported via com public header, hence it is stored as void pointer and be dereferenced with correct internal type in com initialization.
- DIR 1.1: usage of generic function to compare memories. But there is no impact due to this, hence suppressed.
- DIR 1.1: uses generic function independent of the type, and also dereferenced with same type in the target function, hence suppressed.
- RULE 11.5, DIR 1.1 : signalDataPtr_pcv is defined as void pointer in AUTOSAR specification, so suppressing warning “Cast from a pointer to void to a pointer to object”.
- RULE 11.5, DIR 1.1 : signalDataPtr_pcv is defined as void pointer in AUTOSAR specification,so suppressing warning “Cast from a pointer to void to a pointer to object”.
- RULE 11.5: For SignalDataPtr is defined as void pointer in the AUTOSAR specification, so suppressing warning “Cast from a pointer to void to a pointer to

object"

- RULE 11.5: For signalDataPtr_pv is defined as void pointer in the AUTOSAR specification, so suppressing warning "Cast from a pointer to void to a pointer to object"
- RULE 11.5: For signalDataPtr_pv is defined as void pointer in the AUTOSAR specification, so suppressing warning "Cast from a pointer to void to a pointer to object".
- RULE 11.5: On assignment, target pointer will reference to the memory address, and there are no side effects. Hence suppressed the warning.
- RULE 11.5: data read by type casting to "Com_SigMax_tuo" is used in only in case reference data is of that type, otherwise its data is not used. Hence, suppressed.
- RULE 11.5: pointer is referenced with correct type, hence suppressed.
- RULE 11.5: signalDataPtr_pcv is defined as void pointer in AUTOSAR specification, so suppressing warning "Cast from a pointer to void to a pointer to object".
- RULE 11.5: signalDataPtr_pcv is defined as void pointer in the AUTOSAR specification, so suppressing warning "Cast from a pointer to void to a pointer to object".
- RULE 11.5: signalDataPtr_pv is defined as void pointer in the AUTOSAR specification, to support different groupsignal types. On call to this api, application has to pass a valid datatype pointer address. Hence this warning can safely be ignored.
- RULE 11.5: signalDataPtr_pv is defined as void pointer in the AUTOSAR specification, to support different groupsignal types. On call to this api, application has to pass a valid datatype pointer address.Hence this warning can safely be ignored.
- RULE 12.3: "SchM_CData" contents are generated based on "RTE-VFB trace" configuration in RTE. Hence the warning, but it has no functional impact, so, it is suppressed.
- RULE 13.5, DIR 1.1: Explicit cast is provided in line with COM_MEMCOMPARE definition. But the void pointer is always deferenced to a type, based on the pointers alignment. Hence the alignment warning can safely be ignored. Below conditional statement access RAM, use of intermediate variables to store and read here, may read obsolete value. Evaluation(or no evaluation) shall not impact the system behaviour.Hence suppressed.
- RULE 13.5: Not invoking Com_Prv_isValidRxIpdu_b function for unsuccessful confirmation has no impact, hence warning is suppressed.
- RULE 13.5: Below conditional statement access RAM, use of intermediate variables to store and read here, may read obsolete value. Evaluation(or no evaluation) of 2nd operand shall not impact the system behaviour. Hence suppressed.
- RULE 13.5: Below conditional statement access RAM, use of intermediate variables to store and read here, may read obsolete value. Evaluation(or no

evaluation) shall not impact the system behaviour. Hence suppressed.

- RULE 13.5: Below conditional statement reads flag status. Evaluation (or no evaluation) of the 2nd operand does not impact the system behavior, hence the warning is suppressed.
- RULE 13.5: Below conditional statement reads flag status. Evaluation (or no evaluation) of the 2nd or 3rd operands does not impact the system behaviour even though of volatile-qualified, moreover HIS-LEVEL parameter will be violated if more conditional statements are added. Hence the warning is suppressed.
- RULE 13.5: Below conditional statement reads flag status. Evaluation(or no evaluation) of operands does not impact the system behaviour even though of volatile-qualified, Hence the warning is suppressed.
- RULE 13.5: Below conditional statement reads flag status. Evaluation(or no evaluation) of the 2nd operand does not impact the system behaviour. moreover HIS-LEVEL parameter will be violated if more conditional statements are added. Hence the warning is suppressed.
- RULE 13.5: Below conditional statement reads flag status. Evaluation(or no evaluation) of the 2nd or 3rd operands does not impact the system behaviour, moreover HIS-LEVEL parameter will be violated if more conditional statements are added. Hence the warning is suppressed.
- RULE 13.5: Code is designed such that Com_Prv_IsInvalidGrpSigData() needs to be conditionally executed, so, there is no impact independent whether Com_Prv_IsInvalidGrpSigData() is executed/not, hence suppressed.
- RULE 13.5: Code is designed such that Com_Prv_IsInvalidSigData() needs to be conditionally executed, so, there is no impact independent whether Com_Prv_IsInvalidSigData() is executed/not, hence suppressed.
- RULE 13.5: Optional evaluation of the second operand of the below logical-AND does not impact the system behaviour, as it accesses only constant memory. Hence suppressed.
- RULE 2.8: In case no PredefinedVariant is referenced from EcucPostBuildVariants, one public C structure with the name Com_Config shall be generated by the ECUC processor. Hence unused structure warning is suppressed.
- RULE 8.13: A pointer parameter(RetryInfoPtr) in a function prototype should be declared as pointer to const if the pointer is not used to modify the addressed object. But AUTOSAR SWS specification signature is as below, Hence suppressed
- RULE 8.13: A pointer parameter(SignalDataPtr) in a function prototype should be declared as pointer to const if the pointer is not used to modify the addressed object. But AUTOSAR SWS specification signature is as below, hence suppressed
- RULE 8.13: A pointer parameter(signalDataPtr_pv) in a function prototype should be declared as pointer to const if the pointer is not used to modify

the addressed object. But AUTOSAR SWS specification signature is as below, hence suppressed

- RULE 8.13: A pointer parameter(signalGroupArrayPtr_pu8) in a function prototype should be declared as pointer to const if the pointer is not used to modify the addressed object. But AUTOSAR SWS specification signature is as below, hence suppressed
- RULE 8.13: This is a false positive warning, 'signalDataPtr_pv' is modified and so the pointer cannot be changed to type 'pointer to const'. Hence warning is suppressed.

2.4.2 ComM (Communication Manager)

- DIR 1.1: Com_ReceiveSignal's SignalDataPtr return value should be void as per Autosar guidelines.
- DIR 1.1: Com_SendSignal's SignalDataPtr return value should be void as per Autosar guidelines.
- DIR 1.1: NvM_ReadBlock's NvM_DstPtr return value should be void as per Autosar guidelines.
- DIR 1.1: NvM_WriteBlock's NvM_SrcPtr return value should be void as per Autosar guidelines.
- RULE 1.2: Parameter COMM_MAX_CHANNEL_REQUESTERS is generated with different value based on configuration and is not always 1. Reordering the member element ComM_InternalHandleArrayType would potentially impact the functionality. Since the warning has no memory misalignment, no functional impact and not critical, this warning can be ignored.
- RULE 10.3: An expression of 'essentially Boolean' type (_Bool) is being converted to unsigned type, 'unsigned char' on assignment. This is a Incorrect Misra Warning. This is taken care in review.
- RULE 11.3: The pointer type cast to channel specific UserHandleArrayType structure type is necessary as the object has to be of the same structure type declared in the RTE. There would be no misalignment as the channel specific UserHandleArrayType structures and its structure elements would fetch the corresponding memory. Hence this will not impact any functionality.
- RULE 11.5:void pointer is been converted to pointer to ComM_GlobalConfigType_tst before using
- RULE 12.3: "Rte_Switch_UM" generated contents are based on "RTE-VFB trace". This has no functional impact, hence the warning is suppressed.
- RULE 14.2: The variable incremented in the third expression of this 'for' statement is actually the variable identified as the 'loop control variable. Hence it is an incorrect warning reported by Misra
- RULE 14.3: The value of COMM_GET_FEATURE_PNC_ENABLED depends on the configuration therefore it will not always be true.
- RULE 14.3:'if' controlling expression is a constant expression and its value is

'true'. This expression is not true always. This expression value gets updated in the channel state machine for every main function. Hence it is an incorrect warning reported by Misra

- RULE 8.13: The PncEventStruct_ps pointer variables are modified in main function. Hence it cant be P2CONST type
- RULE 8.13: The PncEventStruct_pst pointer variables are modified in main function. Hence it cant be P2CONST type
- RULE 8.13: The pncRamPtr_ps pointer variables are modified in main function. Hence it cant be P2CONST type
- RULE 8.13: The requestStructPtr_pst pointer variables are modified in main function. Hence it cant be P2CONST type
- RULE 8.13:Violated since SWS gives function definition in the given way
- RULE 8.2: Parameter identifiers missing in declaration of a function type and will not impact on functionality
- RULE 8.3, 8.13: 8.3 Two declarations exists for this api. One is in ComM.h & another generated by RTE. Hence, the difference in identifiers. 8.13 AUTOSAR SWS specification signature is as below, hence suppressed
- RULE 8.3:Different <Bus>SM's use different parameters. Hence it is not possible to solve this warning
- RULE 8.3:Two declarations exists for this api. One is in ComM.h & another generated by RTE. Hence,the difference in identifiers.
- RULE 8.5, 8.6: Multiple declarations of external object or function. The function declaration is also generated by the RTE if it is configured. These APIs are accessible by BSW component(BswM). If the declaration is not provided by ComM.h then build would fail
- RULE 8.5: Multiple declarations of external object or function. The function declaration is also generated by the RTE if it is configured. These APIs are accessible by BSW component(BswM). If the declaration is not provided by ComM.h then build would fail
- RULE 8.6: The warning is not critical and has no functional impact. The function definition is available when COMM_VERSION_INFO_API is STD_ON and this macro is available in private header file. The warning is considered as LOW_PRIO

2.4.3 ComXf (ComBased Transformer)

- DIR 1.1: Implicit conversion from a pointer to an object type, to a pointer, to void is necessary to do array copy. Warning is uncritical
- DIR 1.1: Since the last argument passed to the function is configuration specific, uniqueness of first 63 characters cannot be assured. But it doesn't lead to any issue. Warning is uncritical
- DIR 1.1: since the last argument passed to the function is configuration specific, uniqueness of first 63 characters cannot be assured. But it doesn't

lead to any issue. Warning is uncritical

- RULE 10.1, 10.8, 10.3, 10.4: Signed to unsigned conversion is required for serialization of sint signals as per AUTOSAR. Bitwise operation is necessary to fetch/align the correct bit positions for the groupSignals. Warning is uncritical
- RULE 10.3, 10.1, 10.8: As per AUTOSAR requirement, conversion to a different type is necessary in this case. It is ensured that the assigned value is not exceeding the required Size. Warning is uncritical
- RULE 10.3, 11.8: As per AUTOSAR requirement, conversion to a different type is necessary in this case. It is ensured that the assigned value is not exceeding the required Size. Warning is uncritical. Buffer has to hold non constant values which results in loss of const qualification. Warning is uncritical
- RULE 10.3: As per AUTOSAR requirement, conversion to a different type is necessary in this case. It is ensured that the assigned value is not exceeding the required Size. Warning is uncritical
- RULE 11.3: Casting to an object pointer type (uint8). It is the fastest way to write data to unaligned adresses. The alternative is multiple bitshifts and mask operations + uint8 typecast. Warning is uncritical
- RULE 11.3: Pointer is cast to stricter alignment as the rba_XfrmEnableAlignedBufferAccess parameter is set to true which guarantees that the start of the buffer is aligned. Warning is uncritical
- RULE 12.2: Shift operator is used to get the required byte from the element to store in configured byte order. Warning is uncritical

2.4.4 IpduM (I-Pdu Multiplexer)

- DIR 1.1: Explicit cast is provided in line with rba_BswSrv_MemCopy() definition. But the void pointer is always deferenced to a type based on the pointers address alignment. Hence the alignment warning can safely be ignored.
- RULE 13.5: IpduM_Prv_GetTxContainedType returns the Type of Container. It will not have any functional impact.
- RULE 13.5: The api IpduM_Prv_ReadIdFromHeader reads the Header Id of the Contained Pdu from Container. Checking it with zero is to check valid HeaderId is configured. Below Statement will not have any Side effect.
- RULE 8.13: This is a false positive warning, The parameter 'PduInfoPtr' is provided by Autosar so the pointer cannot be changed to type 'pointer to const'. Hence warning is suppressed.

2.4.5 LdCom (Efficient COM for Large Data)

- DIR 1.1: Warning supressed intentionally- the pointer being typecasted is of the matching type, No memory allignment issues expected

- RULE 1.1: As mentioned in Cchecker Known Issues this warning is considered as False Positive and this will not have any impact on functionality
- RULE 11.5: pointer to void is being cast to a pointer to object matching type. The conversion will not result in any misalignment and hence supressed

2.4.6 Mirror

- RULE 13.5: The return value of Mirror_Prv_ChkAndUpdateRelDataTimeStmp API boolean along with that the function also updates the timeStamp to a Global struct variable. No functional impact. Hence the warning is supressed.
- RULE 8.13: A pointer to structure (Mirror_SrcNetworkVarType_tst) in a function prototype should be declared as pointer to const if the pointer is not used to modify the addressed object. But as per our design this is made as RAM struct and this warning will have no functional impact.Hence suppressed

2.4.7 Nm (Network Manageemnt Interface)

- DIR 1.1: Com_SendSignal's SignalDataPtr return value should be void as per Autosar guidelines.

2.4.8 PduR (PDU Router)

- DIR 1.1: Warning supressed intentionally- the pointer being typecasted is of the matching type, No memory allignment issues expected
- RULE 1.3,13.2 : No issue in code.
- RULE 11.5 : 'pointer-to-void' is being cast to a 'pointer-to-object' type and this conversion will not result in any misalignment. Hence Suppressed.
- RULE 11.5 : 'pointer-to-void' is being cast to a 'pointer-to-object' type and this conversion will not result in any misalignment. Hence Supressed.
- RULE 11.5, 11.8 : 'pointer-to-void' is being cast to a 'pointer-to-object' type and this conversion will not result in any misalignment. Hence Supressed.
- RULE 11.5,11.8 : 'pointer-to-void' is being cast to a 'pointer-to-object' type and this conversion will not result in any misalignment. Hence Supressed.
- RULE 11.5: Cast from a pointer to void to a pointer to object type is required to be in sync with the target (PduR_Base) type. Hence this warning can be safely suppressed
- RULE 20.7: Below check is required. In the definition of a function-like macro each instance of a parameter shall be enclosed in parentheses unless it is used as the operand.
- RULE 20.7: Putting 's' in (), throws compilation error. So cannot remove MISRA warning.
- RULE 20.7: The Type Conversion is Required
- RULE 8.13: A pointer parameter in a function prototype should be declared as

pointer to const if the pointer is not used to modify the addressed object. But AUTOSAR SWS specification signature is as below, Hence suppressed

- RULE 8.13: AUTOSAR specifies that pointer should be a P2VAR as it can be changed during run time
- RULE 8.13: As per AUTOSAR SWS specification signature is as made as P2VAR. So this warning can be ignored.
- RULE 8.13: The api is deprecated hence no functional impact. The api will be removed in upcoming release.
- RULE 8.13: The api is internally used by PduR when upper layer Trigger Transmit api is not configured hence there is no functional impact with the warning
- RULE 8.13: The member elements of dyn pointer is modified only if STORE_AND_FORWARD feature is enabled. There warning is senn only when this feature is disabled.

2.4.9 rba_ComStackCdd (Rba Comstack Complex Device Driver)

No violations

2.4.10 rba_XCoreCdd

- DIR 1.1 RULE 11.5: The cast from void to uint8 is intended by design. Rational: in order to avoid potential mistakes due to pointer arithmetic with ptr to an object type we prefer having an explicit void ptr.
- DIR 1.1 RULE 11.8,11.5: The pointer cast is required by design.
- DIR 1.1 RULE 21.15: 1. Explicit cast is provided in line with rba_BswSrv_MemCopy() definition 2. This type casting cannot be avoided as library function expects
- DIR 1.1, RULE 10.3: Cast is desired at this position in order to keep anonymous object type
- RULE 1.1,10.3: Currently configuration doesn't allow activation of Runnable task to read PIPE
- RULE 1.1,10.3: Currently configuration doesn't allow activation of SW IRQ to read PIPE
- RULE 1.1,11.5,11.8: The pointer cast is required by design.
- RULE 10.4,11.5,11.8: The pointer cast is required by design.
- RULE 11.3,11.8: The cast is needed by design. The code is trying to write a packet header, important for the communication protocol Loss of volatile is not critical in this context.
- RULE 11.5: The cast is needed by design. The packet pointer is a volatile shared RAM area
- RULE 11.5: The pointer cast is required by design.
- RULE 13.2,10.3: The reading from volatile counters is intended by design.

Re-read of volatile is OK.

- RULE 13.2: The reading from volatile counters is intended by design. Re-read of volatile is OK.
- RULE 14.3: The DET are used against configuration errors. So the IF() is ok to be constant.
- RULE 16.4: This case is not possible and the comment was written to avoid MISRA warning.
- RULE 20.7: This macro "SCHM_RBA_XCORECDD_GETCURRENT_CORE_ID" expansion is GetCoreID which belongs to "Os.h" file and thus GetCoreID expression is not parenthesized hence the warning occurs. So, no functional impact.
- RULE 8.13: A pointer parameter(RetryInfoPtr) in a function prototype should be declared as pointer to const if the pointer is not used to modify the addressed object. But AUTOSAR SWS specification signature is as below, Hence suppressed

2.4.11 rba_Xfrm

- RULE 11.3: The warning is non-removable since casting is required to manipulate the data as uint16/uint32 type. The warning is uncritical since the data size is reduced and therefore the alignment is even better than required. Approved by MISRA2012_SUPPORT_TEAM.
- RULE 11.3: The warning is non-removable since casting is required to manipulate the data as uint8/uint16 type. The warning is uncritical since the data size is reduced and therefore the alignment is even better than required. Approved by MISRA2012_SUPPORT_TEAM.
- RULE 11.4,11.6: The warning is uncritical and non-removable since casting is required to manipulate memory address as uint32. Approved by MISRA2012_SUPPORT_TEAM.
- RULE 11.5, 11.8: The warning is uncritical and non-removable since casting is required to manipulate memory address as uint8
- RULE 11.5: The warning is non-removable since pointer to void is required to have same syntax as C90 interface and casting is required to manipulate the data. The warning is uncritical since the address range and proper alignment is ensured by the calculation done before in integer representation. Approved by MISRA2012_SUPPORT_TEAM.

2.5 RTA-DIAG

2.5.1 Dcm (Diagnostic Communication Manager)

- DIR 1.1:The type of parameter will be known during runtime, so typecast to void is required.
- RULE 1.3,DIR 4.1: dataAllowedSecRead_u32 will always have the allowed security level mask value. Hence NULL_PTR won't be dereferenced

- RULE 1.3,DIR 4.1: dataAllowedSessRead_u32 will always have the allowed session mask value. Hence NULL_PTR won't be dereferenced
- RULE 1.3: A function-like macro shall not be invoked without all of its arguments, hence some of the arguments are optional based on configuration
- RULE 1.3: RxPduId passed is a valid PduId. So Dcm_Prv_GetMainConnection() returns MainConnection address related to RxPduId. Hence NULL_PTR won't be dereferenced
- RULE 1.3: RxPduId passed is a valid PduId. So Dcm_Prv_GetProtocolRow() returns Protocol address related to RxPduId. Hence NULL_PTR won't be dereferenced
- RULE 10.4: The result of the arithmetic operation will be always in the range of uint16 .Therefore the warning can be ignored
- RULE 10.5: Cast from boolean to uint32 done inntentionally and neccessary according to the current design
- RULE 10.5: Cast from boolean to uint8 done intentionally and neccessary to forward the boolean object in to an uint8 array for the diagnostic response
- RULE 10.8, 10.1: Cast to uint8 and Shift operation is performed on a sint16 object intentionally and neccessary to forward the sint16 object in to an uint8 array for the diagnostic response
- RULE 10.8, 10.1: Cast to uint8 and Shift operation is performed on a sint32 object intentionally and neccessary to forward the sint32 object in to an uint8 array for the diagnostic response
- RULE 10.8: Cast from sint16 to uint8 done intentionally and neccessary to forward the sint16 object in to an uint8 array for the diagnostic response
- RULE 10.8: Cast from sint32 to uint8 done intentionally and neccessary to forward the sint32 object in to an uint8 array for the diagnostic response
- RULE 10.8: Cast from sint8 to uint8 done intentionally and neccessary to forward the sint8 object in to an uint8 array for the diagnostic response
- RULE 11.1 Dcm_GetSeed_ptr1: Typecast required for implementation - Required for efficient RAM usage by using single void function pointer
- RULE 11.1: Typecast required for implementation - Required for efficient RAM usage by using single void function pointer
- RULE 11.1: Typecast to function pointer required for implementation - Required for efficient RAM usage by using single void function pointer
- RULE 11.1: Typecast to function pointer required for implementation Required for efficient RAM usage by using single void function pointer
- RULE 12.2: Right hand operand of shift operator is greater than or equal to the width of the underlying type
- RULE 12.3: Rte interfcace is an auto generated #define which uses a comma operator
- RULE 13.4, 12.3: Rte interfcace is an auto generated #define which requires this construct and uses a comma operator

- RULE 13.5: Right hand operand of '&&' or 'II' is an expression with possible side effects. It is expected, then if one of the conditions returns false right side of && operator would not be executed.
- RULE 13.5: The right hand operand of '&&' or 'II' has side effects - Since all request indexes(0 to DCM_CFG_MAX_REQUESTSINPARALLEL) needs to be checked this condition is required
- RULE 13.5: The right hand operand of '&&' or 'II' has side effects - The statements inside needs to be executed if any of the three conditions are satisfied in this exact order
- RULE 13.5: The right hand operand of '&&' or 'II' has side effects - The statements inside needs to be executed only if all the conditions are satisfied
- RULE 14.2: The implementation needs a loop control variable of file scope
- RULE 14.2: The implementation needs modification of the loop index.
- RULE 15.4 :More than one 'break' statement has been used to terminate this iteration statement.
- RULE 15.4 :More than one 'break' statement has been used to terminate this iteration statement. MISRA C:2012 Rule-15.4
- RULE 15.4: More than one break statement is being used to terminate an iteration statement.
- RULE 15.4:The More than one break statements is required for the implemenatation
- RULE 2.2 : The function is redundant with no side effects and this required for implementation. MISRA C:2012 Rule-2.2
- RULE 21.18,18.1,DIR 4.1 : adrDest_pu8 will never be out of bound as the MsgLen is either 2 to 8bytes(DTC) or 3bytes(DID) and the size of adrDest_pu8 is 8bytes in case of DTC and 4bytes in case of DID
- RULE 21.18,18.1,DIR 4.1 : if index_u16 > 0u granted that index_u16-1u will not cause out of bound access for Dcm_DynamicSignalConfig_cast
- RULE 8.13: The function signature is based on the AR Standard and hence cannot be modified
- RULE 8.13: The object addressed by NegRespCode_u8 will be modified depending on different configurations
- RULE 8.13: The object addressed by negativeResponseCode will be modified depending on different configurations
- RULE 8.13: The object addressed by notificationData will be modified depending on different configurations
- RULE 8.13: The object addressed by pointer are modified under a particular usecase, and hence should be P2VAR
- RULE 8.13: The object addressed by pointer idxDidIndexType_st is modified under a particular usecase, and hence should be P2VAR
- RULE 8.13: The object addressed by pointer pMsgContext is modified under a particular usecase, and hence should be P2VAR

- RULE 8.13: The object addressed by pointer rdbiDataStructure is modified under a particular usecase, and hence should be P2VAR
- RULE 8.13: The object addressed by pointer targetBuffer is being modified by the particular Read functions, and hence it should be P2VAR
- RULE 8.13: The pointers cannot be made constants since MsgPtr is been assigned to adrDest_pu8 and is being modified
- RULE 8.13: The pointers cannot be made constants since they are being modified in other configurtions
- RULE 8.13: The usage of parameter pMsgContext depend on configuration, it will be updated in APCE concept
- RULE 8.13: The usage of parameter pMsgContext is configuration dependent and hence should be P2VAR
- RULE 8.13: cast may discard const qualifier. So used as non constant pointer
- RULE 8.13: pointer needed for async client server use port
- RULE 8.5:This is required for providing Dcm_GetActiveProtocol declaration to rba_DiagAdapt RTE AR4.0 enabled configuration
- RULE 8.5:This is required for providing Dcm_GetSecurityLevel declaration to rba_DiagAdapt RTE AR4.0 enabled configuration
- RULE 8.5:This is required for providing Dcm_GetSesCtrlType declaration to rba_DiagAdapt RTE AR4.0 enabled configuration
- RULE 8.5:This is required for providing Dcm_ResetToDefaultSession declaration to rba_DiagAdapt RTE AR4.0 enabled configuration
- RULE 8.5:This is required for providing Dcm_TriggerOnEvent declaration to rba_DiagAdapt RTE AR4.0 enabled configuration

2.5.2 Dem (Diagnostic Event Manager)

- DIR 1.1: A fast and resource optimized support of different debounce startegies is only possible through this mechanism
- DIR 1.1: Address pointer is declared as void pointer as it is required for mem copy function
- DIR 1.1: Array is generated, always initialized and cannot be empty.
- DIR 1.1: Warning depends on the used configuration. In the used test configuration, it is intentionally done so to check for a specific functionality. In real projects, such configurations will not occur
- DIR 1.1: a fast and resource optimized support of different debounce startegies is only possible through this mechanism
- RULE 1.2, 10.4: 1.2: Array is generated and initialized only if it is not empty (DTCs maps to eventId) 10.4: Variable is defined the struct Dem_MapDtcIdToEventIdType as uint16, no coversion issue
- RULE 1.2: Parameter DEM_STORAGECONDITION_COUNT is generated with different value based on configuration. This warning can be ignored.

- RULE 1.2: The compiler treats the array of length 1 as the final member. But a change of the sequence leads to padding, because different values are based on configuration. This warning can be ignored.
- RULE 1.3, 20.7: 1.3 : A function-like macro shall not be invoked without all of its arguments, hence some of the arguments are optional based on configuration. 20.7: The MACRO is expanded into several commans/lines, this cannot be encapsulated with braces.
- RULE 1.3, 20.7: 1.3: A function-like macro shall not be invoked without all of its arguments, hence some of the arguments are optional based on configuration. 20.7: Functions like MACROS do not need to be parenthesized.
- RULE 1.3: A function-like macro shall not be invoked without all of its arguments, hence some of the arguments are optional based on configuration
- RULE 1.3: For all caller functions, input parameter is not a NULL pointer
- RULE 10.3: Converted value is always positive and does not exceed maximum range of unsigned char
- RULE 10.3: due to the invalid index beeing used the first row includes values not matching the datatype, this row is never used
- RULE 10.4: Evaluation of size of data type, such a size is always constant. Therefore the warning can be ignored
- RULE 10.4: The array type and also the init values of Dem_EvtCategoryPrioType and DEM_EVENTCATPRIO_ARRAYLENGTH are unsigned. The MISRA warning is not justified.
- RULE 10.4: Variable is defined the struct Dem_J1939MemIDMapToDtcIdType as uint16, no coversion issue
- RULE 10.8: No prblem seen in assignment. uint16 value is assigned to uint16 variable.
- RULE 11.3: Casting to different object pointer type because of RTE adaption. This warning can be ignored.
- RULE 11.3: Function defined according to AR Spec in NVM - Std_ReturnType NvM_WriteRamBlockFromNvm(void NvMBuffer)
- RULE 11.5: a fast and resource optimized support of different debounce startegies is only possible through this mechanism
- RULE 11.5: conversion to void pointer needed for memcpy
- RULE 13.2: The order of access of volatile objects is not significant. Therefore this warning can be ignored.
- RULE 13.5, 20.7: 13.5: Boolean returning function is identified as an expression causing side effect. This warning can be ignored. 20.7: Macro parameter FUNCTIONNAME in DEM_COMPONENT_CHECK_EVENTS_ATTRIBUTE may not be enclosed in (), because the MACRO is expanded into several commans/lines.
- RULE 13.5, 20.7: 13.5: Boolean returning function is identified as an expression causing side effect. This warning can be ignored. 20.7: Macro parameter

FUNCTIONNAME may not be enclosed in (), because the MACRO is expanded into several commans/lines.

- RULE 13.5: Boolean returning function is identified as an expression causing side effect. This warning can be ignored.
- RULE 13.5: Getter function is identified as an expression causing side effect. This warning can be ignored.
- RULE 13.5: The operator is free from persistent side effects. Use of #pragma is not allowed.
- RULE 13.5: To improve software quality, the logical expression is encapsulated with getter function. Qac is incapable of recognizing, that a getter function just retrieves information and does not manipulate any data. Therefore it produces false warning.
- RULE 13.5: to improve software quality and mantainability, the bit-manipulation operations of element-attributes are encapsulated in getter/setter functions. Qac is incapable of recognizing, that a getter function just retrieves information and does not manipulate any data. therefore it produces lots of false warnings.
- RULE 14.2: Ring buffer implementation needs a loop control variable of file scope
- RULE 14.2: the parameters might be unused due to the configuration the variable Dem_EventIterator has a file scope but controls through Dem_EvtTriggerUpdateOfNextReportRelevantForMemories and Dem_EvtUpdateNextReportRelevantForMemories in Clear functionality state machine. It shall be declared that way as Dem_EvtUpdateNextReportRelevantForMemories can run over many dem cycles to be completed
- RULE 14.3: Based on the configuration, the loop may or may not execute
- RULE 14.3: First operand is based on configuration and therefore is not a constant.
- RULE 14.3: Operand value depends on configuration and is not a constant.
- RULE 14.3: The operand depends on configuration and is not a constant.
- RULE 2.7: Based on the config of the macro using the parameter ApiId it might not be used in the function
- RULE 2.7: Based on the config of the macro using the parameter ApiId, it might not be used in the function
- RULE 2.7: Parameter internalEnvData will be used in the function based on the configuration. This warning can be ignored.
- RULE 20.7 : Macro parameter may not be enclosed in ().
- RULE 20.7, 10.4: 20.7: Functions like MACROS do not need to be parenthesized. 10.4: The Array type and also the init values of Dem_EvtCategoryPrioType, DEM_EVENTCATPRIO_ARRAYLENGTH and DEM_CFG_EVTCATEGORYPRIO are unsigned. The MISRA warning is not justified.
- RULE 20.7, DIR 1.1: 20.7: Functions like MACROS do not need to be parenthe-

sized.1.1: Address pointer is declared as void pointer as it is required for mem copy function.

- RULE 20.7: Due to the fact that array content and array size are both generated based on the same input it is ensured that the data always fit in to the array
- RULE 20.7: Functions like MACROS do not need to be parenthesized.
- RULE 20.7: Macro parameter FUNCTIONNAME in DEM_COMPONENT_CHECK_EVENTS_ATTRIBUTE may not be enclosed in (), because the MACRO is expanded into several commans/lines.
- RULE 20.7: Macro parameter FUNCTIONNAME in DEM_COMPONENT_CHECK_EVENTS_ATTRIBUTE may not be enclosed in ().
- RULE 20.7: The MACRO is expanded into several commans/lines, this cannot be encapsulated with braces.
- RULE 20.7: the makro is expanded into several commans/lines, this cannot be encapsulated with braces
- RULE 8.13: A pointer parameter in a function prototype should be declared as pointer to const if the pointer is not used to modify the addressed object
- RULE 8.13: Function defined according to AR Spec IndicatorStatus not changed if OBD is OFF
- RULE 8.13: Parameter fe not made const, as it is modified in subfunction
- RULE 8.13: The object addressed by pointer will be changed based on configuration, so it cannot be made constant. This warning can be ignored.
- RULE 8.13: The pointer will be modified based on the configuration.
- RULE 8.13: The tool does not recognize that the pointer is used in a memcopy macro and data is written, this is a tool issue
- RULE 8.13: The warning can be ignored since the interface is not completely implemented
- RULE 8.13: changing value might be needed in customer project
- RULE 8.13: parameter evBuff not made const, as it is modified in subfunction
- RULE 8.13: parameter historyStatus not made const, as it is modified by Dem_EvtGetHistoryStatus() subfunction
- RULE 8.13: parameter not made const, as it is depending on compiler switch setting
- RULE 8.13: parameter not made const, as it is used to get address of element data and to fill the data block
- RULE 8.13: parameter src not made const, as it is modified in subfunction
- RULE 8.13: the parameters might be unused due to the configuration
- RULE 8.3: due to the RTE there is an additional declaration with P2VAR for this function, it is ensured that the internal declarations are correct
- RULE 8.5: Duplicate of Dem_GetDebouncingOfEvent to make it also available in the GeneralDiagnosticInfo RTE interface

- RULE 8.5: Duplicate of Dem_GetEventExtendedDataRecord to make it also available in the GeneralDiagnosticInfo RTE interface
- RULE 8.5: Duplicate of Dem_GetEventExtendedDataRecord without the BufSize parameter to provide a standard-compliant RTE interface
- RULE 8.5: Duplicate of Dem_GetEventExtendedDataRecordEx to make it also available in the GeneralDiagnosticInfo RTE interface
- RULE 8.5: Duplicate of Dem_GetEventFreezeFrameData to make it also available in the GeneralDiagnosticInfo RTE interface
- RULE 8.5: Duplicate of Dem_GetEventFreezeFrameData without the BufSize parameter to provide a standard-compliant RTE interface
- RULE 8.5: Duplicate of Dem_GetEventFreezeFrameDataEx to make it also available in the GeneralDiagnosticInfo RTE interface
- RULE 8.5: Duplicate of Dem_GetEventUdsStatus to make it also available in the GeneralDiagnosticInfo RTE interface
- RULE 8.5: Duplicate of Dem_GetMonitorStatus to make it also available in the GeneralDiagnosticInfo RTE interface
- RULE 8.5: the function declaration is also generated by the RTE if it is configured, this does not cause any issues

2.5.3 Dlt (Diagnostic Log and Trace)

- RULE 1.2: Array size 1 is needed by RTE to generate pointer to uint8
- RULE 10.3: It is ensured that these values are always small enough. Therefore the warning can be ignored
- RULE 17.12: This is intended for the funtion to be generated and not as an expression of pointer type.
- RULE 17.2: The Dlt_DetForwardErrorTrace processes Det errors to DLT other than DLT module itself to avoid recursion ensured by module ID check
- RULE 8.13: Parameter cannot be made as constant since it depends upon the value of DLT_CFG_GENERAL_RX_SUPPORT in different configurations.
- RULE 8.13: Parameter is a complex data type. MISRA can not identify that array is provided as call by value. Function call has no side effects.
- RULE 8.13: Parameter not made const as defined by AUTOSAR
- RULE 8.13: Parameter not made const as function is a private function and only be called locally
- RULE 8.13: Parameter not made const as the sub-functionality will modify the parameter
- RULE 8.13: Parameters appId, contextId cannot be made as const since as it is defined in the Autosar specification
- RULE 8.13: Parameters appId, contextId cannot be made as const since it is defined in the Autosar specification
- RULE 8.5: Duplicate of Dlt_GetDefaultLogLevel to make it available in the

RTE interface

- RULE 8.5: Duplicate of Dlt_GetLogChannelNames to make it available in the RTE interface
- RULE 8.5: Duplicate of Dlt_GetLogChannelThreshold to make it available in the RTE interface
- RULE 8.5: Duplicate of Dlt_GetLogInfo to make it available in the RTE interface
- RULE 8.5: Duplicate of Dlt_RegisterContext to make it available in the RTE interface
- RULE 8.5: Duplicate of Dlt_ResetToFactoryDefault to make it available in the RTE interface
- RULE 8.5: Duplicate of Dlt_SendLogMessage to make it available in the RTE interface
- RULE 8.5: Duplicate of Dlt_SetDefaultLogLevel to make it available in the RTE interface
- RULE 8.5: Duplicate of Dlt_SetLogChannelAssignment to make it available in the RTE interface
- RULE 8.5: Duplicate of Dlt_SetLogChannelThreshold to make it available in the RTE interface
- RULE 8.5: Duplicate of Dlt_SetLogLevel to make it available in the RTE interface
- RULE 8.5: Duplicate of Dlt_SetMessageFiltering to make it available in the RTE interface
- RULE 8.5: Duplicate of Dlt_StoreConfiguration to make it available in the RTE interface

2.5.4 FiM (Function Inhibition Manager)

- RULE 8.5: Duplicate of FiM_GetFunctionPermission to make it also available in the GeneralDiagnosticInfo RTE interface
- RULE 8.5: Duplicate of FiM_MainFunction to make it also available in the GeneralDiagnosticInfo RTE interface

2.5.5 FiM_PBL

No violations

2.5.6 FiM_PBS

No violations

2.5.7 FiM_PC

No violations

2.5.8 rba_DemObdBasic

- DIR 1.1: Value is converted to unsigned type for representation to user outside Dem. This has no side effect.
- DIR 4.9: MACRO is more efficient than function in this case
- RULE 10.3: It is ensured that these values are always small enough, the L is necessary because of other MISRA warnings in the DEM
- RULE 10.4: It is ensured that this conversion does not cause any issues, the definition is necessary in this way because it would otherwise cause more MISRA warnings in other places
- RULE 11.5: cast to void necessary for memcopy operation
- RULE 13.5: Boolean returning function is identified as an expression causing persistent side effect. This warning can be ignored.
- RULE 13.5: The function "rba_DemObdBasic_EvMem_GettFFTimeId" does not create any side effects
- RULE 13.5: The function "rba_DemObdBasic_Event_IsRequestingMil" returns a boolean value, it does not cause any side effects
- RULE 13.5: The function rba_DemObdBasic_OBDonUDS_GetFFTimeId does not create any sideeffects
- RULE 13.5: the boolean function "rba_DemObdBasic_Event_GetMil-RepEvent" does not have any persistent side effects
- RULE 13.5: the function "FiM_GetService07Visibility" returns value of type Std_ReturnType, it does not cause any persistent side effects
- RULE 13.5: the function "rba_DemObdBasic_EvMem_IsDtcConfirmed" does not have any persistent side effects
- RULE 13.5: the function "rba_DemObdBasic_Mil_IsEvMemLocRequesting-Mil" does not have any persistent side effects
- RULE 13.5: the functions "rba_DemObdBasic_EvMem_IsDtcConfirmed","r-ba_DemObdBasic_EvMem_GetMilCounter","Dem_EvMemIsStored" do not have any persistent side effects
- RULE 14.3: Depending on the configuration the loop controlling expression may be 'false'
- RULE 20.7: Function like MACROS need not be parenthesized
- RULE 20.7: Initialization MACRO need not be parenthesized
- RULE 8.13: MISRA does not recognize the manipulation of "dest" pointer through the DEM_MEMCPY macro
- RULE 8.13: MISRA does not recognize the manipulation of DestBuffer through the DEM_MEMCPY macro
- RULE 8.13: The parameter won't be set to const because depending on configuration, the object pointed by the pointer may be modified.
- RULE 8.13: parameter not made const, as it is defined by AUTOSAR

- RULE 8.13: the out parameters will be modified if precondition is satisfied, these cannot be set to const
- RULE 8.13: the parameter Obdmidvalue will be modified if precondition is satisfied, these cannot be set to const
- RULE 8.13: the parameter PID21value will be modified if precondition is satisfied, these cannot be set to const
- RULE 8.13: the parameter PID31value will be modified if precondition is satisfied, these cannot be set to const
- RULE 8.13: the parameter PID4Dvalue will be modified if precondition is satisfied, these cannot be set to const
- RULE 8.13: the parameter PID4Evalue will be modified if precondition is satisfied, these cannot be set to const
- RULE 8.13: the parameter destBuffer and destSize will be modified if precondition is satisfied, these cannot be set to const
- RULE 8.13: the parameter numberOfTIDs will be modified if precondition is satisfied, these cannot be set to const
- RULE 8.13: the parameter won't be set to const because MISRA does not recognise the manipulation of the pointer through the DEM_MEMCPY macro
- RULE 8.13: the parameter won't be set to const because this function is defined this way by AUTOSAR
- RULE 8.5: the function declaration is also generated by the RTE if it is configured, this does not cause any issues
- RULE 9.1: the variable "eventId" is intialized within the function "rba_DemObdBasic_EvMem_GetEventIdAndStatusOfReaderCopy"

2.6 RTA-ETH

2.6.1 DoIP (Diagnostic Over IP)

- DIR 1.1: Function rba_BswSrv_MemCompare converts argument 1 and 2 to pointer to void, function has no side effects
- DIR 1.1: Function rba_BswSrv_MemCopy converts argument 1 and 2 to pointer to void, function has no side effects
- DIR 1.1: Function rba_BswSrv_MemSet converts argument 1 to pointer to void, function has no side effects
- RULE 14.3: The controlling expression is not invariant, since the value of DOIP_CFG_DHCP_OPTION_VIN_USE is derived from configuration. The construct does not use preprocessor directive in order to strike a balance between maintainability and resource consumption
- RULE 14.3: The controlling expression is not invariant, since the value of DOIP_CFG_ENTITY_STATUS_MAX_BYTEFIELD_USE is derived from configuration. The construct does not use preprocessor directive in order to strike a balance between maintainability and resource consumption

- RULE 14.3: The controlling expression is not invariant, since the value of DOIP_CFG_FURTHER_ACTION_BYTE_INDEPENDENT is derived from configuration. The construct does not use preprocessor directive in order to strike a balance between maintainability and resource consumption
- RULE 14.3: The controlling expression is not invariant, since the value of DOIP_CFG_HOSTNAME_SIZE_MAX is derived from configuration. The construct does not use preprocessor directive in order to strike a balance between maintainability and resource consumption
- RULE 14.3: The controlling expression is not invariant, since the value of DOIP_CFG_USE_EID_AS_GID is derived from configuration. The construct does not use preprocessor directive in order to strike a balance between maintainability and resource consumption
- RULE 14.3: The controlling expression is not invariant, since the value of DOIP_CFG_VIN_GID_MASTER is derived from configuration. The construct does not use preprocessor directive in order to strike a balance between maintainability and resource consumption

2.6.2 DoIP_PBL

No violations

2.6.3 DoIP_PBS

No violations

2.6.4 DoIP_PC

No violations

2.6.5 EthIf (Ethernet Interface)

- DIR 1.1: The input parameters of rba_BswSrv_MemCopy()
- RULE 11.3: The casting is required as the lHeaderList_pst is stored as the pointer to the next header of type ListElemStructType.
- RULE 8.13: BaudRatePtr is an out argument and the addressed object is modified inside the function, pointer to variable is defined in AUTOSAR for BaudRatePtr. Deviation is not critical as it has no functional impact.
- RULE 8.13: BufferPtr is an out argument and the addressed object is modified inside the function, pointer to variable is defined in AUTOSAR for BufferPtr. Deviation is not critical as it has no functional impact.
- RULE 8.13: CounterPtr is an out argument and the addressed object is modified inside the function, pointer to variable is defined in AUTOSAR for CounterPtr. Deviation is not critical as it has no functional impact.
- RULE 8.13: DataSizePtr and DataAdressPtr are an out argument and the addressed object is modified inside the function, pointer to variable is defined in AUTOSAR for DataSizePtr and DataAdressPtr. Deviation is not critical as it has no functional impact.

- RULE 8.13: DuplexModePtr is an out argument and the addressed object is modified inside the function, pointer to variable is defined in AUTOSAR for DuplexModePtr. Deviation is not critical as it has no functional impact.
- RULE 8.13: LinkStatePtr is an out argument and the addressed object is modified inside the function, pointer to variable is defined in AUTOSAR for LinkStatePtr. Deviation is not critical as it has no functional impact.
- RULE 8.13: MacAddrPtr, SwitchIdxPtr and PortIdxPtr are an out argument and the addressed object is modified inside the function, pointer to variable is defined in AUTOSAR for MacAddrPtr, SwitchIdxPtr and PortIdxPtr. Deviation is not critical as it has no functional impact.
- RULE 8.13: MacLearningModePtr is an out argument and the addressed object is modified inside the function, pointer to variable is defined in AUTOSAR for MacLearningModePtr. Deviation is not critical as it has no functional impact.
- RULE 8.13: MacModePtr is an out argument and the addressed object is modified inside the function, pointer to variable is defined in AUTOSAR for MacModePtr. Deviation is not critical as it has no functional impact.
- RULE 8.13: OrgUniqueIdPtr is an out argument and the addressed object is modified inside the function, pointer to variable is defined in AUTOSAR for OrgUniqueIdPtr. Deviation is not critical as it has no functional impact.
- RULE 8.13: OrgUniqueIdPtr, ModelNrPtr and RevisionNrPtr are an out argument and the addressed object is modified inside the function, pointer to variable is defined in AUTOSAR for OrgUniqueIdPtr, ModelNrPtr and RevisionNrPtr. Deviation is not critical as it has no functional impact.
- RULE 8.13: OrgUniqueIdPtr, ModelNrPtr and RevisionNrPtr are out arguments and the addressed objects are modified inside the function, pointer to variable is defined in AUTOSAR for OrgUniqueIdPtr, ModelNrPtr and RevisionNrPtr. Deviation is not critical as it has no functional impact.
- RULE 8.13: Pointer to variable is defined in AUTOSAR for DataPtr. Deviation is not critical as it has no functional impact.
- RULE 8.13: PortMirrorConfigurationPtr is an out argument and the addressed object is modified inside the function, pointer to variable is defined in AUTOSAR for PortMirrorConfigurationPtr. Deviation is not critical as it has no functional impact.
- RULE 8.13: PortMirrorStatePtr is an out argument and the addressed object is modified inside the function, pointer to variable is defined in AUTOSAR for PortMirrorStatePtr. Deviation is not critical as it has no functional impact.
- RULE 8.13: PortModePtr is an out argument and the addressed object is modified inside the function, pointer to variable is defined in AUTOSAR for PortModePtr. Deviation is not critical as it has no functional impact.
- RULE 8.13: ResultPtr is an out argument and the addressed object is modified inside the function, pointer to variable is defined in AUTOSAR for ResultPtr. Deviation is not critical as it has no functional impact.
- RULE 8.13: RxStatsPtr is an out argument and the addressed object is modi-

fiedinside the function, pointer to variable is defined in AUTOSAR for RxStatsPtr. Deviation is not critical as it has no functional impact.

- RULE 8.13: SpiStatusPtr is an out argument and the addressed object is modified inside the function, pointer to variable is defined in AUTOSAR for SpiStatusPtr. Deviation is not critical as it has no functional impact.
- RULE 8.13: SwitchPortEgressFifoBufferLevelPtr is an out argument and the addressed object is modified inside the function, pointer to variable is defined in AUTOSAR for SwitchPortEgressFifoBufferLevelPtr. Deviation is not critical as it has no functional impact.
- RULE 8.13: TrcvModePtr is an out argument and the addressed object is modified inside the function, pointer to variable is defined in AUTOSAR for TrcvModePtr. Deviation is not critical as it has no functional impact.
- RULE 8.13: TxStatsPtr is an out argument and the addressed object is modified inside the function, pointer to variable is defined in AUTOSAR for TxStatsPtr. Deviation is not critical as it has no functional impact.
- RULE 8.13: WakeupReasonPtr is an out argument and the addressed object is modified inside the function, pointer to variable is defined in AUTOSAR for WakeupReasonPtr. Deviation is not critical as it has no functional impact.
- RULE 8.13: When EthIfCtrl has no VLAN configured, the adressed object by ZeroCopyArg_pst is modified inside the function (resizing of the buffer given in ZeroCopyArg_pst is executed)
- RULE 8.13: numberOfElements and arlTableListPointer are an out argument and the addressed object is modified inside the function, pointer to variable is defined in AUTOSAR for numberOfElements and arlTableListPointer. Deviation is not critical as it has no functional impact.

2.6.6 EthSM (Ethernet State Manager)

No violations

2.6.7 EthTSyn (Time Synchronization over Ethernet)

- DIR 1.1: cast needed by design of rba_BswSrv_MemCopy. No risk.

2.6.8 rba_EthArp

- RULE 2.8: The object rba_EthArp_Config_configName_cs is used during module initialization by Tcplp.

2.6.9 rba_EthAutoIp

- RULE 2.8: The object rba_EthAutoIp_Config_0_cst is used and referenced from Tcplp component, inside Tcplp_ConfigSet structure.

2.6.10 rba_EthDHCP

- RULE 11.3: Casting from a pointer to an AUTOSAR specific data structure type to a pointer to an AUTOSAR generic data structure type and vice versa is safe when the common data structure member occurs first in the data structure and the allocated object of specific data type was correctly identified before its specific data structure members are accessed.

2.6.11 rba_EthDHCPv6

- DIR 1.1: cast needed by design of rba_BswSrv_MemCopy. No risk.
- RULE 1.3, DIR 4.1: The messages REQUEST, RENEW, RELEASE and DECLINE are always built and sent after a DHCPv6 server has been selected. Therefore, adrClientData_pcst→adrSelectedServer_pst is never NULL referenced.
- RULE 1.3, DIR 4.1: The option "Server ID" is added only to messages REQUEST, RENEW, RELEASE and DECLINE which are always built and sent after a DHCPv6 server has been selected. Therefore, adrClientData_pcst→adrSelectedServer_pst is never NULL referenced.
- RULE 11.3: Casting from a pointer to an AUTOSAR specific data structure type to a pointer to an AUTOSAR generic data structure type and vice versa is safe when the common data structure member occurs first in the data structure and the allocated object of specific data type was correctly identified before its specific data structure members are accessed.

2.6.12 rba_EthICMP

- DIR 1.1: The input parameter of Tcplp_OnesComplChkSum() is declared as (void) for faster implmentation of checksum calculation. Low Risk
- RULE 11.3: The cast is safe because the generic type is used by the invoked function and extracts the info depending on the domain.
- RULE 11.3: Typecasted to Tcplp_SockAddrInetType to match the parameter Type of function definition. Cast is safe because the domain is checked to be equal to TCPIP_AF_INET above
- RULE 2.8: The object rba_Ethicmp_Config_0_cst is used and referenced from Tcplp component, inside Tcplp_ConfigSet structure.
- RULE 8.13: The variable Buffer_pu8 is used only when if TCPIP_ONES_COMPL_CHKSUM_ENABLED == STD_ON, Since it is configuration dependent, it can not be changed to const.

2.6.13 rba_EthIcmpV6

- DIR 1.1: The first input parameter of Tcplp_OnesComplChkSum() is declared as (void) for generic implementation of function.
- RULE 11.3: Typecasted to Tcplp_SockAddrInetType to match the parameter

Type of function definition. Cast is safe because the domain is checked to be equal to TCPIP_AF_INET above

- RULE 11.8, DIR 1.1: The first input parameter of Tcplp_OnesComplChkSum() is declared as (void) for generic implementation of function.

2.6.14 rba_EthIPv4

- DIR 1.1: The input parameter of Tcplp_OnesComplChkSum() is declared as (void) for faster implmentation of checksum calculation. Low Risk
- DIR 1.1: The input parameters of rba_BswSrv_MemCopy() are declared as (void) for generic implementation of MemCopy. Low Risk
- RULE 1.3, DIR 4.1: The ReassemblyBufDescriptor_pst pointer is a valid pointer, since this function rba_EthIPv4_FindInitialHolePos is called by rba_EthIPv4_MergeFragIntoReassemblBuf() only if a valid reassembly buffer exists.
- RULE 1.3, DIR 4.1: The lReassemblyBufDescriptor_pst pointer is a valid pointer, because the if loop is entered only when the check of Reassembly-BufCnt_cu8 is not equal to lLoopIdx_u8 passes, indicating the existence of a valid reassembly buffer.
- RULE 1.3, DIR 4.1: The lReassemblyBufDescriptor_pst pointer is a valid pointer, since the else loop is entered only if a valid reassembly buffer is allocated.
- RULE 1.3, DIR 4.1: The lReassemblyBufDescriptor_pst pointer is a valid pointer, since this function rba_EthIPv4_MergeFragIntoReassemblBuf is called by rba_EthIPv4_ReassemblyHandler() only if a valid reassembly buffer exists.
- RULE 15.4: Avoiding multiple break statements will make code unreadable
- RULE 8.13: According to 'Specification of Ethernet Interface AUTOSAR Release 4.1.1', in EthIf_RxIndication() parameter RmtPhysAddr_pcu8 is of pointer to variable.
- RULE 8.13: Buffer_ppu8 is not modified in the static function. But it is modified in the caller function and hence it is retained as pointer to variable.

2.6.15 rba_EthIPv6

- DIR 1.1: The input parameters of rba_BswSrv_MemCopy() are declared as (void) for generic implementation of MemCopy. Low Risk
- RULE 1.3, DIR 4.1: The ReassemblyBufDescriptor_cpcst pointer is a valid pointer, since this function rba_EthIPv6_FindInitialHolePos is called by rba_EthIPv6_MergeFragIntoReassemblBuf() only if a valid reassembly buffer exists.
- RULE 1.3, DIR 4.1: The lReassemblyBufDescriptor_pst pointer is a valid pointer, since the else loop is entered only if a valid reassembly buffer is allocated.

- RULE 1.3, DIR 4.1: The lReassemblyBufDescriptor_pst pointer is a valid pointer, since this function rba_EthIPv6_MergeFragIntoReassemblBuf is called by rba_EthIPv6_ReassemblyHandler() only if a valid reassembly buffer exists.
- RULE 11.3, RULE 1.3, DIR 4.1: The 2nd and 3rd arguments passed are pointers to arrays of uint32[4] , but have different datatypes (structure names). Typecasting is safe as data size and type remains same, The lReassembly-BufDescriptor_pst pointer is a valid pointer, since the check of Reassembly-BufCnt_cu8 not equal to lcheck_buffer_config_u8, indicates the existence of a valid reassembly buffer.
- RULE 11.3: Cast is provide to match the data types exactly. Its safe because size of both remain same.
- RULE 11.3: Cast is provided to match the data types exactly. Its safe because size of both remain same.
- RULE 11.3: Cast is the only way to dynamically provide to Tcplp_GetIpAddr theIP address structure which is domain independant. It is safe because the casted structure type is smaller than the declared structure type.
- RULE 11.3: The 2nd and 3rd arguments passed are pointers to arrays of uint32[4] , but have different datatypes (structure names). Typecasting is safe as data size and type remains same.
- RULE 11.3: The 3rd argument passed and destination arguments are pointers to arrays of uint32[4] , but have different datatypes (structure names).
- RULE 11.3: The 3rd argument passed and destination arguments pointers to arrays of uint32[4] , but have different datatypes (structure names). Typecasting is safe as data size and type remains same.
- RULE 11.3: The argument passed is pointer to array of uint32[4] , but have different datatypes (structure names). Typecasting is safe as data size and type remains same.
- RULE 13.5,11.3:Cast is provide to match the data types exactly. Its safe because size of both remain same.
- RULE 15.4: Avoiding multiple break statements will make code unreadable
- RULE 15.4: Multiple break statements has kept the code effecient and readable. Code has been reviewed and tested , it is safe
- RULE 15.4: The break instruction is used for speed optimization. Code is reviewed and safe.
- RULE 2.8: The object rba_EthIPv6_configName_cst is supposed to be used during module initialization by Tcplp

2.6.16 rba_EthNdp

- RULE 11.3: Cast is provide to match the data types exactly. Its safe because size of both remain same.
- RULE 11.3: Cast is the only way to dynamically provide to Tcplp_GetIpAddr

the IP address structure which is domain independant. It is safe because the casted structure type is smaller than the declared structure type.

- RULE 13.5,11.3: Cast is provide to match the data types exactly. Its safe because size of both remain same.
- RULE 13.5,11.3:Cast is provide to match the data types exactly. Its safe because size of both remain same.
- RULE 13.5: Function argument is type const unit32 whereas , actual argument is of type uint32[4]. Typecasting to make datatypes same. Conversion is safe as size remains same and additional cont is added .
- RULE 2.8: The object rba_EthNdp_Config_0_cst is used and referenced from Tcplp component, inside Tcplp_ConfigSet structure.
- RULE 8.13: INeighRoutAdv_st cannot be of type pointer to constant as it's modification is dependent on TCPIP_NDP_PRD_PREFIXROUTERDISC_ENABLED compiler switch

2.6.17 rba_EthTcp

- DIR 1.1: The input parameters of Tcplp_OnesComplChkSum() are declared as (void) for generic implementation.
- DIR 1.1: The input parameters of rba_BswSrv_MemCopy() are declared as (void) for generic implementation of MemCopy.
- DIR 1.1: The input parameters of rba_BswSrv_MemCopy() are declared as (void) for generic implementation of MemCopy. Low Risk
- DIR 4.1: Out of bound cannot happen because array index was checked before.
- RULE 1.3: Out of bound cannot happen because array index was checked before.
- RULE 11.3 : Cast is required to respect the type definition of the calling function. Typecasted to Tcplp_SockAddrType to be used in domain agnostic function call. It is safe because of the domain switch
- RULE 11.3 Typecasted to Tcplp_SockAddrType to be used in domain agnostic function call. It is safe because of the domain switch
- RULE 11.3, 1.3, 4.1 : Cast is the only way to dynamically interpret the data as IPv4 or IPv6 specific data. Typecasted to Tcplp_SockAddrType to be used in domain agnostic function call. It is safe because of the domain switch
- RULE 11.3, 1.3, 4.1: Cast is the only way to dynamically interpret the data as IPv4 or IPv6 specific data. Typecasted to Tcplp_SockAddrType to be used in domain agnostic function call. It is safe because of the domain switch
- RULE 11.3: Cast is required to respect the type definition of the calling function. Cast is safe because the domain is checked to be equal to TCPIP_AF_INET above
- RULE 11.3: Cast is required to respect the type definition of the calling function. Typecasted to Tcplp_SockAddrType pointer to be used in domain

agnostic function call. It is safe because of the domain switch

- RULE 11.3: Cast is required to respect the type definition of the calling function. Typecasted to Tcplp_SockAddrType to be used in domain agnostic function call. It is safe because of the domain switch
- RULE 11.3: Cast is required to respect the type definition of the calling function. Typecasted to Tcplp_SockAddrType to be used in domain agnostic function call. It is safe because of the domain switch.
- RULE 11.3: Cast is the only way to dynamically interpret the data as IPv4 or IPv6 specific data.
- RULE 11.3: Cast is the only way to dynamically interpret the data as IPv4 or IPv6 specific data. Cast is safe because the domain is checked to be equal to TCPIP_AF_INET above
- RULE 11.3: Cast is the only way to dynamically interpret the data as IPv4 or IPv6 specific data. Cast is safe because the domain is checked to be equal to TCPIP_AF_INET6 above
- RULE 11.3: Cast is the only way to dynamically interpret the data as IPv4 or IPv6 specific data. It is safe because domain is checked before typecasting in the macro expansion
- RULE 11.3: Cast is the only way to dynamically interpret the data as IPv4 or IPv6 specific data. Typecasted to Tcplp_SockAddrType to be used in domain agnostic function call. It is safe because of the domain switch
- RULE 11.3: Casting to different object pointer type. Typecasted to Tcplp_-SockAddrType to be used in domain agnostic function call. It is safe because of the domain switch
- RULE 11.3: Typecasted to Tcplp_SockAddrType to be used in domain agnostic function call. It is safe because of the domain switch.
- RULE 11.5: cast needed by design of TCP. The input parameters of Tcplp_TcpTransmitMetaData() which invokes rbe_EthTcp_Transmit()are declared as (void pointer) for generic implementation. Typecasting is safe because only first 4 bytes of data pointed by MetaDataPtr_pv is required here
- RULE 11.5: cast needed by design of TCP. The input parameters of Tcplp_TcpTransmitMetaData() which invokes rbe_EthTcp_Transmit()are declared as (void pointer) for generic implementation. Typecasting is safe because only first 4 bytes of data pointed by MetaDataPtr_pv need update
- RULE 18.1: Out of bound cannot happen because array index was checked before.
- RULE 2.8 : The object rba_EthTcp_Config_cst is used and referenced from Tcplp component, inside Tcplp_ConfigSet structure.

2.6.18 rba_EthUdp

- DIR 1.1: The input parameter of Tcplp_OnesComplChkSum() is declared as (void) for faster implmentation of checksum calculation. Low Risk

- DIR 1.1: The input parameters of rba_BswSrv_MemCopy() are declared as (void) for generic implementation of MemCopy.
- RULE 1.3, DIR 4.1: It is ensured that lBuffer_pu8 and llpBuffer_pu8 are valid because the return code from rba_EthUdp_ProvideTxBuffer() is checked before executing the mathematical operation on pointers. When rba_EthUdp_ProvideTxBuffer() returns BUFREQ_OK, that means that lower layer (IPv4 or IPsec) returned valid pointers.
- RULE 10.4: Type definition of wildcard IP address is according to AUTOSAR [SWS_TCPIP_00133].
- RULE 11.3: Cast is the only way to dynamically interpret the data as IPv4 data. It is safe because of the domain check before.
- RULE 11.3: Cast is the only way to dynamically interpret the data as IPv4 data. It is safe because of the switch before.
- RULE 11.3: Cast is the only way to dynamically interpret the data as IPv6 data. It is safe because of the switch before.
- RULE 11.3: Cast is the only way to dynamically provide to Tcplp_GetIpAddr the IP address structure which is domain independant. It is safe because the casted structure type is smalled than the declared structure type.
- RULE 11.3: Typecasted to Tcplp_SockAddrType to be used in domain agnostic function call. It is safe because of the domain switch.
- RULE 2.8: The object rba_EthUdp_Config_0_cst is used and referenced from Tcplp component, inside Tcplp_ConfigSet structure.

2.6.19 Sd (Service Discovery)

- DIR 1.1: Structure Name is configuration dependent so it might be greater then 63 in some cases
- DIR 1.1: cast needed by design of rba_BswSrv_MemCopy. No risk.
- RULE 10.5: The cast is safe because it is not causing wrong value assignment as the storage is using a derived type
- RULE 11.3: The cast is safe because the generic type is used by the invoked function and extracts the info depending on the domain
- RULE 15.4: More than one break used as there are multiple valid if condition where loop should break.
- RULE 15.4: More than one break used as these breaks are there in independent if conditions
- RULE 5.2: Identifier does not differ from other identifier(s) within the specified number of significant characters
- RULE 8.13: The pointers cannot be made constants as it will be used when eventgroups are not zero
- RULE 8.4: Datastructure is defined without any previous declaration, since this is used in the same file

2.6.20 SoAd (Socket Adapter)

- DIR 1.1: The input parameters of Tcplp_TcpTransmitMetaData() are declared as (void) for generic implementation
- DIR 1.1: The input parameters of Tcplp_UdpTransmitMetaData() are declared as (const void) for generic implementation of Tcplp_UdpTransmitMetaData
- DIR 1.1: The input parameters of rba_BswSrv_MemCopy() are declared as (void) for generic implementation of MemCopy.
- RULE 11.3: Cast is required to respect the type definition of the calling MACRO. Typecasted to Tcplp_SockAddrType to be used in domain agnostic MACRO. It is safe because of the domain switch.
- RULE 11.3: Cast is required to respect the type definition of the calling function. Typecasted to Tcplp_SockAddrType to be used in domain agnostic function call. It is safe because of the domain switch.
- RULE 11.3: The cast is safe because the MetadataTyperef is checked above for Socket connection Id
- RULE 11.3: The cast is safe because the generic type is used by the caller function and extracts the info depending on the domain.
- RULE 11.3: Typecasted to Tcplp_SockAddrInet6Type to match the local parameter Type definition. Cast is safe because the domain is checked to be equal to TCPIP_AF_INET6 above.
- RULE 11.5: cast needed by design of TCP . No risk.
- RULE 11.8: The cast is safe because even though SduDataPtr is variable overall PduInfoType that is passed to the UL is still const.
- RULE 14.3: The Value of the conditional expression is always false and its not a critical issue
- RULE 8.13: Parameter is not const uint8 according to AR specification of this API
- RULE 8.13: Not critical, P2VAR is defined in AUTOSAR for length
- RULE 8.13: SockAddr_pst is modified inside the function and hence cannot be P2CONST
- RULE 8.13: The Pointer SockAddrInet6_pst is not used when only IPV4 is ON, but will be used when both IPV4 and IPV6 are ON
- RULE 8.13: The Pointer SockAddrInet_pst is not used when only IPV6 is ON, but will be used when both IPV4 and IPV6 are ON

2.6.21 SomeIpTp

No violations

2.6.22 SomeIpXf (SOME/IP Transformer)

- DIR 1.1: The generated variable name is based on the configuration. There is

no functionality impact due to the warning. Hence it is uncritical.

- DIR 1.1: Warning is not critical and implicit conversion from a pointer to object type to a pointer to void is necessary, to do array copy. Warning is uncritical
- RULE 1.3: Null pointer check is perfomed initiallyand so correct ponter will get dereferenced. Hence the warning is uncritical
- RULE 10.3, 1.3: Rule-10.3 Conversion is necessary in this case to convert to uint16 type as per AUTOSAR requirement. Its ensured that assigned value is not exceeding the required size. Rule-1.3 Null pointer check is perfomed initiallyand so correct ponter will get dereferenced. Hence the warning is uncritical
- RULE 10.3, 1.3: Rule-10.3 Conversion is necessary in this case to convert to uint8 type as per AUTOSAR requirement. Its ensured that assigned value is not exceeding the required size. Rule-1.3 Null pointer check is perfomed initially and so correct ponter will get dereferenced. Hence the warning is uncritical
- RULE 10.3, 10.4 : Conversion is necessary in this case to convert to uint8 type as per AUTOSAR requirement. Its ensured that assigned value is not exceeding the required size.Warning is uncritical
- RULE 10.3, 10.4: The conversion of unsigned int to singned int is necessary in this case
- RULE 10.3, 10.4: The conversion of unsigned int to singned int is necessary to meet AUTOSAR requirement. Its ensured that assigned value is not exceeding the required size. Warning is uncritical
- RULE 10.3, 10.4: The generation of data type is configuration dependent.
- RULE 10.3, 10.4: The generation of data type is configuration dependent. However, If length field of data type is of narrower unsigned type, then the buffer offset will also not exceed. Warning is uncritical
- RULE 10.3: Assignment of zero to the structure members is required to initialize the structure object
- RULE 10.3: Conversion is necessary in this case to convert the type as per AUTOSAR requirement. Its ensured that assigned value is not exceeding the required Size. Warning is uncritical
- RULE 10.3: Conversion is necessary in this case to convert to uint16 type as per AUTOSAR requirement. Its ensured that ${tabs}assigned value is not exceeding uint16 size
- RULE 10.3: Conversion is necessary in this case to convert to uint16 type as per AUTOSAR requirement. Its ensured that assigned value is not exceeding the BufferOffset Size. Warning is uncritical
- RULE 10.3: Conversion is necessary in this case to convert to uint16 type as per AUTOSAR requirement. Its ensured that assigned value is not exceeding the required size. Warning is uncritical
- RULE 10.3: Conversion is necessary in this case to convert to uint8 type as per AUTOSAR requirement

- RULE 10.3: Conversion is necessary in this case to convert to uint8 type as per AUTOSAR requirement. Its ensured that assigned value is not exceeding the Datatype Size. Warning is uncritical
- RULE 10.3: Conversion is necessary in this case to convert to uint8 type as per AUTOSAR requirement. Its ensured that assigned value is not exceeding the required size. Warning is uncritical
- RULE 10.3: Conversion is necessary in this case to convert to uint8 type as per AUTOSAR requirement. Its ensured that assigned value is not exceeding the required size.Warning is uncritical
- RULE 10.3: The conversion of signed int to unsigned char type is necessary in this case
- RULE 10.3: The conversion of unsigned int to unsingned short is necessary in this case
- RULE 10.3: The conversion of unsigned long to unsigned short is necessary in this case
- RULE 10.3: The conversion of unsigned long to unsingned int is necessary in this case
- RULE 10.3: The conversion to unsigned char is necessary in this case
- RULE 10.3: The generation of data type is configuration dependent. Conversion is necessary in this case to convert the type as per AUTOSAR requirement. Its ensured that assigned value is not exceeding the required Size. Warning is uncritical
- RULE 10.3: The generation of data type is configuration dependent. However, If length field is data type is of narrower unsigned type, then the buffer offset will also not exceed. Warning is uncritical
- RULE 10.3: The generation of data type is configuration dependent. However, If length field of data type is of narrower unsigned type, then the buffer offset will also not exceed. Warning is uncritical
- RULE 10.4: Conversion is necessary in this case to convert to uint8 type as per AUTOSAR requirement
- RULE 10.4: Conversion is necessary in this case to convert to uint8 type as per AUTOSAR requirement. Its ensured that assigned value is not exceeding the required size.Warning is uncritical
- RULE 10.4: The generation of data type is configuration dependent
- RULE 10.4: The generation of data type is configuration dependent. Conversion is necessary in this case to convert the type as per AUTOSAR requirement. Its ensured that assigned value is not exceeding the required Size. Warning is uncritical
- RULE 16.3: As per code design, non-matched value of the case should return error. As function is returned here, break statement not required and warning is uncritical
- RULE 16.3: As per code desing, non-mathced value of the case, it should return error hence, default does not have break;

- RULE 16.3: The final switch clause must end with a jump statement i,e return of E_SER_MALFORMED_MESSAGE instead of break to indicate that the De-serialization was unsuccessfull.Warning is uncritical.
- RULE 16.3: The final switch clause must end with a jump statement i,e return of E_SER_MALFORMED_MESSAGE instead of break to indicate that the serialization was unsuccessfull.Warning is uncritical.
- RULE 8.13: The function is generated to be compliant with RTE call function. Hence pointer to var is used instead of Pointer to const
- RULE 8.13: The function is generated to be compliant with RTE call function. Hence pointer to var is used instead of Pointer to const. Warning is uncritical

2.6.23 Tcplp (TCP/IP Stack)

- RULE 1.3, RULE 18.1, DIR 4.1, DIR 1.1: When ZeroCopy mode type is TCPIP_ZEROCOPY_DYNAMIC_MODE, there are at least (1 Reuse header + 1 IpOneTime header + 1 UdpOneTimeHeader) allocated, therefore it is not possible to have AllocatedHeaderCount_u8 being underflown and therefore the array access will be always valid.
- RULE 11.3: Typecasted to Tcplp_SockAddrInet6Type to match the local parameter Type definition. Cast is safe because the domain is checked to be equal to TCPIP_AF_INET6 above.
- RULE 11.3: Typecasted to Tcplp_SockAddrInetType to match the local parameter Type definition. Cast is safe because the domain is checked to be equal to TCPIP_AF_INET above.
- RULE 11.3: Typecasted to Tcplp_SockAddrInetType to match the parameter Type of function definition. Cast is safe because the domain is checked to be equal to TCPIP_AF_INET above.
- RULE 11.3: Typecasted to Tcplp_SockAddrType to match the local parameter Type definition.Cast is safe because the domain is checked to be equal to TCPIP_AF_INET6 above.
- RULE 11.3: typecasted to Tcplp_SockAddrInetType to match the local parameter Type definition
- RULE 11.5: Cast uint16 pointer is required for faster checksum calculation. It is done on the Tx/Rx buffer that should be 2 byte aligned
- RULE 15.4: The break instruction is used for speed optimization.
- RULE 2.8: The object Tcplp_ConfigSet is used during module initialization. EcuM calls Tcplp_Init() with pointer to the Tcplp ConfigSet.
- RULE 8.13: IpAddrPtr and DefaultRouterPtr are out arguments and the addressed objects are modified inside the function, pointer to variable is defined in AUTOSAR for IpAddrPtr and DefaultRouterPtr. Deviation is not critical as it has no functional impact.
- RULE 8.13: PhysAddrPtr is an out argument and the addressed object is modified inside the function, pointer to variable is defined in AUTOSAR for PhysAddrPtr. Deviation is not critical as it has no functional impact.

- RULE 8.13: Pointer to variable is defined in AUTOSAR for DataLength and DataPtr. Deviation is not critical as it has no functional impact.

2.6.24 UdpNm (UDP Network Management)

- RULE 2.8: The object UdpNm_ConfigDataÂ< <variantNameÂ> >_cs is used during module initialization. EcuM calls UdpNm_Init() with pointer to the UdpNm ConfigData.
- RULE 8.13: RamPtr_ps is changed inside the called function and hence can not be made 'pointer to const'

2.7 RTA-ETHSEC

2.7.1 rba_EthAh

- DIR 1.1: The first input parameter of rba_BswSrv_MemCopy() is declared as (void) for generic implementation of function.
- DIR 1.1: The first input parameter of rba_BswSrv_MemSet() is declared as (void) for generic implementation of function.
- RULE 11.3: Casting from a pointer to an AUTOSAR specific data structure type to a pointer to an AUTOSAR generic data structure type and vice versa is safe when the common data structure member occurs first in the data structure and the allocated object of specific data type was correctly identified before its specific data structure members are accessed.
- RULE 11.3: Casting from a pointer to an AUTOSAR specific data structure type to a pointer to an AUTOSAR generic data structure type and vice versa is safe when the common data structure member occurs first in the data structure and the allocated object of specific data type was correctly identified before its specific data structure members are accessed.

2.7.2 rba_EthIke2

- RULE 1.3, DIR 4.1: It is ensured through previous checks that the correct input parameters are provided to the rba_EthIke2_Prv_GetPayload() API, so that it returns a valid buffer adrPaylNi_pu8 with sufficient buffer length at this point of code execution.
- RULE 1.3, DIR 4.1: It is ensured through previous checks that the correct input parameters are provided to the rba_EthIke2_Prv_GetPayload() API, so that it returns a valid buffer adrPaylNotif_pu8 with sufficient buffer length at this point of code execution.
- RULE 1.3, DIR 4.1: It is ensured through previous checks that the correct input parameters are provided to the rba_EthIke2_Prv_GetPayload() API, so that it returns a valid buffer adrSk_pu8 with sufficient buffer length at this point of code execution.
- RULE 1.3, DIR 4.1: It is ensured via validation scripts that rba_EthIke2_Prv_GetEncryBlockSize() API returns a non zero value at this

point of code(The encryption transform will always be RBA_ETHIKE2_ENCRYTFM_AES_CBC).

- RULE 11.3: Casting from a pointer to an AUTOSAR specific data structure type to a pointer to an AUTOSAR generic data structure type and vice versa is safe when the common data structure member occurs first in the data structure and the allocated object of specific data type was correctly identified before its specific data structure members are accessed.

2.7.3 rba_EthIPsec

- RULE 1.3, DIR 4.1: When function parameter adrLocalEnd_pcst is NULL then also the parameters adrLocalStart_pcst, adrRemoteStart_pcst and adrRemoteEnd_pcst are NULL and idProtProt_en is RBA_ETHIPSEC_PROTPROT_NONE and idSecuProt_en is RBA_ETHIPSEC_SECUPROT_NONE. In this case, the first OR-operand is TRUE, and the second OR-operand will not be evaluated. Therefore, the function parameter adrLocalEnd_pcst will not be dereferenced when it is a NULL pointer.
- RULE 1.3, DIR 4.1: When function parameter adrLocalStart_pcst is NULL then also the parameters adrLocalEnd_pcst, adrRemoteStart_pcst and adrRemoteEnd_pcst are NULL and idProtProt_en is RBA_ETHIPSEC_PROTPROT_NONE and idSecuProt_en is RBA_ETHIPSEC_SECUPROT_NONE. In this case, the first OR-operand is TRUE, and the second OR-operand will not be evaluated. Therefore, the function parameter adrLocalStart_pcst will not be dereferenced when it is a NULL pointer.
- RULE 1.3, DIR 4.1: When function parameter adrRemoteStart_pcst is NULL then also the parameters adrLocalStart_pcst, adrLocalEnd_pcst and adrRemoteEnd_pcst are NULL and idProtProt_en is RBA_ETHIPSEC_PROTPROT_NONE and idSecuProt_en is RBA_ETHIPSEC_SECUPROT_NONE. In this case, the first OR-operand is TRUE, and the second OR-operand will not be evaluated. Therefore, the function parameter adrRemoteStart_pcst will not be dereferenced when it is a NULL pointer.
- RULE 11.3: Casting from a pointer to an AUTOSAR specific data structure type to a pointer to an AUTOSAR generic data structure type and vice versa is safe when the common data structure member occurs first in the data structure and the allocated object of specific data type was correctly identified before its specific data structure members are accessed.
- RULE 11.3: Casting from a pointer to an AUTOSAR specific data structure type to a pointer to an AUTOSAR generic data structure type and vice versa is safe when the common data structure member occurs first in the data structure and the allocated object of specific data type was correctly identified before its specific data structure members are accessed.

2.7.4 rba_EthTls

- DIR 1.1: Structure name depends on the short name of configured certificate, so it can be exceeded the max allowable length if short name is too big. Since it is not related to any functional impact, treated as No risk.

- DIR 1.1: Structure name depends on the short name of configured ciphersuite worker, so it can be exceeded the max allowable length if short name is too big. Since it is not related to any functional impact, treated as No risk.
- DIR 1.1: Structure name depends on the short name of configured cipher-suite, so it can be exceeded the max allowable length if short name is too big. Since it is not related to any functional impact, treated as No risk.
- DIR 1.1: Structure name depends on the short name of configured hand-shake, so it can be exceeded the max allowable length if short name is too big. Since it is not related to any functional impact, treated as No risk.
- DIR 1.1: Structure name depends on the short name of configured psk iden-tity, so it can be exceeded the max allowable length if short name is too big. Since it is not related to any functional impact, treated as No risk.
- DIR 1.1: The input parameters of rba_BswSrv_MemCopy() are declared as (void) for generic implementation of MemCopy.
- DIR 4.1 : Access outside the bounds of an array is not possible because because the condition check is done before in if statement. This warning can be ignored
- RULE 11.3: Typecasted to Tcplp_SockAddrInetType to match the local parameter Type definition. Cast is safe because the domain is checked to be equal to TCPIP_AF_INET above.
- RULE 11.3: Typecasted to Tcplp_SockAddrInetType to match the local parameter Type definition. Cast is safe because the domain is checked to be equal to TCPIP_AF_INET6 above.
- RULE 21.18,18.1: Access outside the bounds of an array is not possible because the condition check is done before in if statement. This warning can be ignored
- RULE 8.13: Dst_pu8 will be updated inside the called function rba_BswS-rv_MemCopy, so it can't be changed to const pointer. Since it is related to only qualifying pointer, it is treated as No Risk.

2.8 RTA-FRAY

2.8.1 FrIf (FlexRay Interface)

- RULE 10.3: Fr_ClrBit is a MACRO that clears the bit at a given position in FrIf message buffer. Thus the typecasting is intentional in order to clear the correct bit position.
- RULE 10.3: Fr_ClrBit is a MACRO that clears the bit at a given position in FrIf message buffer. Thus the typecasting is intentional in order to clear the correct bit position.
- RULE 10.3:Fr_ClrBit is a MACRO that clears the bit at a given position in FrIf message buffer. Thus the typecasting is intentional in order to clear the correct bit position.
- RULE 10.3:Fr_SetBit is a MACRO that sets the bit at a given position in FrIf

message buffer. Thus the typecasting is intentional in order to set the correct bit position.

- RULE 10.5, 10.8: Fr_GetBit is a MACRO that gets the bit at a given position in FrIf message buffer. Thus the typecasting is intentional in order to get the correct bit position.
- RULE 10.5,10.8:Fr_GetBit is a MACRO that gets the bit at a given position in FrIf message buffer. Thus the typecasting is intentional in order to get the correct bit position.
- RULE 14.2: This warning can be ignored since this operation is intentionally implemented for proper indexing of arrays elements.
- RULE 14.3: This warning can be ignored since this operation is intentionally implemented to report development errors. hence the controlling expression is not needed.
- RULE 14.3: This warning can be ignored since this operation is intentionally implemented; hence the controlling expression is not needed
- RULE 2.4: The structure is used in FrIf.c
- RULE 2.4: The structure is used in FrIf_PbCfg.c
- RULE 5.9: This is a local variable and does not have any impact since no linkage
- RULE 7.3: A lowercase letter L (l) has been used in an integer or floating suffix.

2.8.2 FrNm (FlexRay Network Management)

- DIR 1.1: Pointer to object is converted to pointer to void to protect the contents of the FrNm_Priv.h file to get exported to other modules. Hence suppressed.
- RULE 11.5: Cast from a pointer to void to a pointer to object type is done to protect the contents of the FrNm_Priv.h file to get exported to other modules. Hence suppressed.
- RULE 16.3: The preceding switch case has break statement only for one of the conditions, because the FrNm should remain in bus sleep state (shall not immediately switch to NM_MODE_SYNCHRONIZE) if network is not requested by passive or active wakeup
- RULE 2.2: This warning is applicable only if configured FrNmPnInfoLength is 1. The value of the FrNmPnInfoLength parameter can vary. Hence suppressed.
- RULE 8.13: Data in RamPtr_ps is overwritten When FrNmSynchronizedPnc-ShutdownEnabled is enabled, No functional impact because of this warning
- RULE 8.13: Data in RamPtr_ps is overwritten When FrNmSynchronizedPnc-ShutdownEnabled,FrNmPnSyncShutdownErrorReactionEnabled and FrNm-PassiveModeEnabled are enabled, No functional impact because of this warning

2.8.3 FrNm_PB_Variant_Only

No violations

2.8.4 FrNm_PreCompile_and_PB_Variant

No violations

2.8.5 FrSM (FlexRay State Manager)

No violations

2.8.6 FrTp (FlexRay Transport Protocol)

- RULE 1.3,18.1,DIR 4.1 : FrTp_Queue will never be out of bound as it has a fixed value equivalent to the number of configured channels
- RULE 1.3,18.1,DIR 4.1 : FrTp_TxNPduRamData will never be out of bound as it has a fixed value equivalent to the number of configured Tx pdus.
- RULE 10.3: It is ensured that the values are of the same type as defined in PduLengthType which is of type uint16.
- RULE 10.3: It is ensured that the values are of the same type as defined in PduLengthType which is of type uint16. Further after the ternary operator 0 is kept as a signed integer to handle the compiler warnings when the flag [-Werror=sign-compare] is enabled
- RULE 10.3: It is ensured that the values are of the same type as defined in PduLengthType which is of type uint8.
- RULE 13.5: There are no side effects,as boolean function FrTp_Prv_Check-MetadataConn(Connection) is used to verify the return value for the connection that is passed as a parameter, generated from structures. Boolean function FrTp_Prv_ValidateRxMetaData(RxContext, PduInfoPtr) is used to verify the rx metadata is stored properly in sequence. Therefore there is no manipulation of data and the statements inside need to be executed only if all the conditions are satisfied MISRA C:2012 Rule-13.5
- RULE 13.5: There are no side effects,as boolean function FrTp_Prv_Check-MetadataConn(Connection) is used to verify the return value for the connection that is passed as a parameter, generated from structures. PduInfoPtr→MetaDataPtr is simply used to verfiy that the Metadata ptr is null. Therefore there is no manipulation of data and the statements inside need to be executed only if all the conditions are satisfied MISRA C:2012 Rule-13.5
- RULE 18.6: Design implementation and Code optimzation Queue implementation using LinkedLists When data is deleted, the address if the QueueData is modified to point to a new location i.e Queue→Head. MISRA C:2012 Rule-18.6
- RULE 18.6: Design implementation and Code optimzation through the concept of Queue implementation using LinkedList If the destination queue is empty, then the head of the source queue points to the destination's

queue's head. MISRA C:2012 Rule-18.6

- RULE 18.6: Design implementation and Code optimzation through the concept of Queue implementation using LinkedList MISRA C:2012 Rule-18.6
- RULE 18.6: Design implementation and Code optimzation through the concept of Queue implementation using LinkedList The data pointed by Source queue's head is copied to destination queue's head.MISRA C:2012 Rule-18.6
- RULE 18.6: Design implementation and Code optimzation through the concept of Queue implementation using LinkedList When data is added to a queue it is added in the Tail of the queue. Hence we modify the tail to point to the new element MISRA C:2012 Rule-18.6
- RULE 18.6: Design implementation and Code optimzation through the concept of Queue implementation using LinkedList When the head of the queue is pointing to a null location, then the queuedata is added to the head of the queue. Hence the head points to the new element. MISRA C:2012 Rule-18.6
- RULE 18.6: Design implementation and Code optimzation through the concept of Queue implementation using LinkedList. The data pointed by Source queue's tail is copied to destination queue's tail. MISRA C:2012 Rule-18.6
- RULE 2.8 : Since the FrTp_Config_VPOSTBS_Variant_Master_4cyl is passing as an argument to FrTp_Init function, so this structure should availale externally. hence there are no issues
- RULE 5.6,5.7,5.8,5.9 : the elements have been declared here to define the element type and generate the types.h file. hence there are no issues
- RULE 5.7,5.8,5.9 : the elements have been declared here to define the element type and generate the types.h file. hence there are no issues
- RULE 5.8,5.9: the array has been declared inside a unique structure hence, this does not cause any issues
- RULE 5.9 : the elements have been declared here to define the element type and generate the types.h file. hence there are no issues

2.8.7 FrTp_PB_Loadable

No violations

2.8.8 FrTp_PB_Selectable

No violations

2.8.9 FrTp_PreCompile

No violations

2.9 RTA-HWD(Can)

2.9.1 CanTrcv (Can Transceiver)

- RULE 10.3: A non-constant expression of 'essentially unsigned' type (unsigned short) is being converted to narrower unsigned type, 'unsigned char' on assignment. Functional not impacted
- RULE 10.3: A non-constant expression of 'essentially unsigned' type (unsigned short) is being converted to narrower unsigned type, 'unsigned char' on assignment.Functional not impacted
- RULE 11.3: Convertion of OS-Time to CanTrcv_Time. As this is integrator code, different time-values-types has to be mapped to a general CanTr-cv-time type
- RULE 11.3: The buffer-parameter types are given by Autosar as uint8. But HW works with uint16.
- RULE 11.3: The buffer-parametertypes are given by Autosar as uint8. But HW works with uint16.

2.10 RTA-HWD(Eth)

2.10.1 EthTrcv (Ethernet Tranceiver)

- RULE 10.4:EthTrcv_PhyTestModeType is defined in EthIf and its definition needs to be updated there and hence the value here is hardcoded as 7 in the DET check
- RULE 18.1: Buffer index ITrcvIdx_u8 will be taken care in configuration. ITrcvIdx_u8 default value is 0 as only one transceiver is supported so no risk
- RULE 20.1: Required by Autosar memmap strategy
- RULE 20.4: Requirement from Autosar to implement the GetVersion function as macro.

2.10.2 EthTrcv_11_Common

No violations

2.10.3 EthTrcv_11_StdAutomPhy

- RULE 10.4:EthTrcv_PhyTestModeType is defined in EthIf and its definition needs to be updated there and hence the value here is hardcoded as 7 in the DET check
- RULE 10.5: This vio from Platform_Types, the type _Bool is mapped here to the AUTOSAR type boolean to prevent the direct use of the type _Bool
- RULE 10.5: This violation is from Platform_Types, the type _Bool is mapped here to the AUTOSAR type boolean to prevent the direct use of the type _Bool

- RULE 10.5: This violation is from Platform_Types, the type _Bool is mapped here to thetype boolean to prevent the direct use of the type _Bool
- RULE 10.5: This violation istform_Types, the type _Bool is mapped here to the AUTOSAR type boolean to prevent the direct use of the type _Bool
- RULE 10.5: This violfrom Platform_Types, the type _Bool is mapped here to the AUTOSAR type boolean to prevent the direct use of the type _Bool
- RULE 10.ON: This violation is from Platform_Types, the type _Bool is mapped here to the AUTOSAR type boolean to prevent the direct use of the type _Bool
- RULE 20.1: Required by Autosar memmap strategy
- RULE 20.4: Requirement from Autosar to implement the GetVersion function as macro.
- RULE ATION: This violation is from Platform_Types, the type _Bool is mapped here to the AUTOSAR type boolean to prevent the direct use of the type _Bool
- Rule 8.4:It is made sure that declaration of this varibale is avaialable

2.11 RTA-HWD(Fray)

2.11.1 FrTrcv (FlexRay Transceiver Driver)

- RULE 10.5, 10.8, 10.3, 10.1: The reason (enum) is calculated out of the bitmask (uint32) given by hardware.
- RULE 10.5, 10.8: The reason (enum) is calculated out of the bitmask (uint32) given by hardware.
- RULE 11.3, 1.1: The buffer-parametertypes are given by Autosar as uint8. But HW works with uint16.
- RULE 11.3: Convertion of OS-Time to FrTrcv_Time. As this is integrator code, different time-values-types has to be mapped to a general FrTrcv-time type
- RULE 13.4: Depending on used integration code there can be a Misra warning.
- RULE 15.4: This function is very time critical (for some Trcv loop shorter than 1us) so this violation is checked and accepted
- RULE 15.4: Multiple breaks are required to return from function when there is a error.In case of error detection,further action is not required.This is not critical.
- RULE 16.4: In Current implemetation if the caluclated value is invalid then state cannot be updated as there is no unknown state. Hence defualt cannot have any value.
- RULE 5.9: declaration/definition of local variables are present in different component - no issue
- RULE 8.13: This function will internally call the SPI APi where RXBuffer will be updated with received data. hence this cannot be made as constant pointer

.

2.12 RTA-HWD(Lin)

2.12.1 LinTrcv (Lin Tranceiver)

- RULE 11.3: Conversion of OS-Time to LinTrcv-Time. As this is integrator code, different time-values-types has to be mapped to a general LinTrcv-time type

2.13 RTA-J1939

2.13.1 J1939Dcm (J1939 Diagnostic Communication Manager)

- RULE 10.1: Error Indicator is updated based on the Request Command. So arithmetic operation performed on the MemAccessReqCmd of type 'enum' cannot be avoided
- RULE 10.3: MemAccessApplConfCallbackState is updated based on the MemAccessState. So a conversion to a different enum type cannot be avoided
- RULE 10.5, 10.8: Changing the order of operations will affect the functionality.
- RULE 10.5, 10.8: Typecasting to enum type is required for implementation as target container is of type J1939Dcm_MemAcsReqCmdType.
- RULE 11.5: Casting from pointer to void to pointer of type J1939Dcm_MasterConf_Tst is required for proper initialization of the configuration pointer
- RULE 12.2: Changing the order of operations will affect the functionality.
- RULE 13.5: Right hand operand of '&&' or 'II' is an expression with possible side effects. It is expected, then if one of the conditions returns false right side of && operator would not be executed.
- RULE 15.4: Multiple Break statements added intentionally as required for functionality (for each possible return value of DEM API)
- RULE 15.4: Multiple break statements are necessary for this requirement (for each possible return value of DEM API)
- RULE 8.13: MemAcsStructPtr is never used to modify the data which it addresses. It is just used as a read only parameter for condition checks.
- RULE 8.13: The pointer cannot be made constant since MsgContextPtr is being modified
- RULE 8.5: the function declaration is also generated by the RTE if it is configured, this does not cause any issues

2.13.2 J1939Nm (J1939 Network Management)

- DIR 1.1: 'pointer-to-void' is being cast to a 'pointer-to-object' type and this

conversion will not result in any misalignment

- RULE 10.3: Since J1939NM_STATE_PRE_CLAIMING and J1939NM_STATE_-CLAIMING are obtained from J1939Nm_IntNodeChnlStateType_ten, Nm_S-tateType is not consistent with a controlling expression
- RULE 10.3: Since NM_STATE_NORMAL_OPERATION is obtained from Nm_S-tateType, J1939Nm_IntNodeChnlStateType_ten is not consistent with a controlling expression
- RULE 13.5: Below conditional statement has a check for destAddress. It will not have any functional impact, hence the warning is suppressed.
- RULE 8.13: Since J1939Nm_NodeChnlVarPtrType_tpst struct is generated during the run time and it's a var pointer so it is not possible to typecast it to const
- RULE 8.13: Since J1939Nm_NodeChnlVarPtr_pstType struct is generated during the run time and it's a var pointer so it is not possible to typecast it to const

2.13.3 J1939Rm (J1939 Request Manager)

- DIR 1.1: 'pointer-to-void' is being cast to a 'pointer-to-object' type and this conversion will not result in any misalignment
- RULE 10.3: A non-constant expression of 'essentially unsigned' type (unsigned long) is being converted to narrower unsigned type, 'unsigned char' on assignment.
- RULE 10.5: Here, the Ack code which is received will be of uint8 type which has to be assigned to J1939Rm_AckCode enum type variable
- RULE 11.5: On assignment, target pointer will reference to the memory address, and there are no side effects. Hence suppressed the warning.
- RULE 12.3: Macro Name will change based on configuration This MISRA warning will not have any functional impact
- RULE 18.1: The NodeId_u8 which is updated will be of based on User Type Node Value so it attempts to access memory inside the bounds of a valid memory area
- RULE 8.13: A pointer parameter(PduInfoPtr) in a function prototype should be declared as pointer to const if the pointer is not used to modify the addressed object. But AUTOSAR SWS specification signature is as below, Hence suppressed
- RULE 8.13: The pointer MetaDataPtr_tpst is copied from DataStream and it's contents are modified - therefore it cannot be const.
- RULE 8.13: The pointer nodePtr_tpst is copied from DataStream and it's contents are modified therefore it cannot be const
- RULE 8.5, 8.3: This is declared in J1939Rm and also generated in Rte, as this is done on purpose to reduce the dependency between the modules and does not have any functional impact

- Rule 2.7: The ComMChannelId_t,PGN_u32, destAddress variables used for different functionalities separated by a condition #if(J1939RM_TIMEOUT_-SUPERVISION_SUPPORT != STD_OFF), #if(J1939RM_USER_CDD_EN-ABLED != STD_OFF) This are used parameters to the function.
- Rule 2.7: The ComMChannelId_t,PGN_u32, destAddress variables used for different functionalities separated by a condition #if(J1939RM_TIMEOUT_-SUPERVISION_SUPPORT != STD_OFF), #if(J1939RM_USER_RTE_ENABLED != STD_OFF) This are used parameters to the function.
- Rule 2.7: The UserType_en, HandleId_t variables used for different functionalities separated by a condition #if(J1939RM_TIMEOUT_SUPERVI-SION_SUPPORT != STD_OFF), #if(J1939RM_USER_CDD_ENABLED != STD_OFF) This are used parameters to the function.
- Rule 2.7: The UserType_en, HandleId_t variables used for different functionalities separated by a condition #if(J1939RM_TIMEOUT_SUPERVI-SION_SUPPORT != STD_OFF), #if(J1939RM_USER_RTE_ENABLED != STD_OFF) This are used parameters to the function.
- Rule 5.9: The variable used are for different functionalities separated by a condition [#if (J1939RM_RQST_TRANSMISSION_SUPPORT == STD_ON), (J1939RM_ACK_TRANSMISSION_SUPPORT == STD_ON)]. This does not affect the functionality of the module.

2.13.4 J1939Tp (J1939 Transport Protocol)

- RULE 10.8: Division results into tick value which has to be of type uint16. The value J1939Tp_GetRxBamT1TimeoutCalibration will not cross the upper limit of float32 since the configuration doesn't use such big values MISRA C:2012 Rule-18.6
- RULE 10.8: Division results into tick value which has to be of type uint16. The value J1939Tp_GetRxCmdtT1TimeoutCalibration will not cross the upper limit of float32 since the configuration doesn't use such big values.MISRA C:2012 Rule-18.6
- RULE 10.8: Division results into tick value which has to be of type uint16. The value J1939Tp_GetTxBamTrMinWaitTimeCalibration will not cross the upper limit of float32 since the configuration doesn't use such big values MISRA C:2012 Rule-18.6
- RULE 10.8: Division results into tick value which has to be of type uint16. The value J1939Tp_GetTxCmdtTrMaxTimeoutCalibration will not cross the upper limit of float32 since the configuration doesn't use such big values MISRA C:2012 Rule-18.6
- RULE 10.8: Division results into tick value which has to be of type uint16. The value J1939Tp_GetTxCmdtTrMinWaitTimeCalibration will not cross the upper limit of float32 since the configuration doesn't use such big values MISRA C:2012 Rule-18.6
- RULE 13.5: The operator is free from persistent side effects. Use of #pragma is not allowed.

- RULE 18.6: Design implementation and Code optimzation Queue implementation using LinkedLists When data is deleted, the address if the ConnectionData is modified to point to a new location i.e Queue→Head. MISRA C:2012 Rule-18.6
- RULE 18.6: Design implementation and Code optimzation through the concept of Queue implementation using LinkedList If the destination queue is empty, then the head of the source queue points to the destination's queue's head. MISRA C:2012 Rule-18.6
- RULE 18.6: Design implementation and Code optimzation through the concept of Queue implementation using LinkedList MISRA C:2012 Rule-18.6
- RULE 18.6: Design implementation and Code optimzation through the concept of Queue implementation using LinkedList When data is added to a queue it is added in the Tail of the queue. Hence we modify the tail to point to the new element MISRA C:2012 Rule-18.6
- RULE 18.6: Design implementation and Code optimzation through the concept of Queue implementation using LinkedList When the head of the queue is pointing to a null location, then the queuedata is added to the head of the queue. Hence the head points to the new element. MISRA C:2012 Rule-18.6

2.13.5 J1939Tp_PB_Loadable

No violations

2.13.6 J1939Tp_PB_Selectable

No violations

2.13.7 J1939Tp_PreCompile

No violations

2.14 RTA-LIN

2.14.1 LinIf (Lin Interface and Lin Transport Protocol)

- DIR 1.1: Warning supressed intentionally- Void pointer initialised with pointer to an instance of structure LinIf_InternalConfigTypeNo memory allignment issues expected
- DIR 1.1: Warning supressed intentionally- Void pointer initialised with pointer to an instance of structure LinTp_InternalConfigTypeNo memory allignment issues expected
- RULE 10.3: Warning supressed intentionally- Callback function returns with either True(0x01) or False(0x00) So, This MISRA warning will not have any functional impact
- RULE 10.5: Warning suppressed intentionally- the VAR being typecasted is of the matching type, No memory alignment issues expected

- RULE 11.5: Warning supressed intentionally- the pointer being typecasted is of the matching type, No memory allignment issues expected
- RULE 11.8: Warning suppressed intentionally-Called function has no const qualifier
- RULE 11.8: Warning supressed intentionally- VAR is typcasted to meet the functional requirement
- RULE 2.2: Macro expansion of SchM is available from SchM component which is not in scope of LinIf. This MISRA warning will not have any functional impact
- RULE 5.5: Warning supressed intentionally- As per AUTOSAR specification the argument name is Schedule, in Os.h there is a macro which causes this warning
- RULE 8.13: Channel pointer LinTp_ChnlVarPtr_ps is used as VAR in the calling function so in this inline function it cannot be changed to CONST
- RULE 8.13: Warning suppressed intentionally- the VAR is unchanged to comply with AUTOSAR signature
- RULE 8.13: Warning supressed intentionally- the pointer is unchanged to comply with AUTOSAR signature
- RULE 8.3:Callback prototype used is same for all upper layers.

2.14.2 LinSM (Lin State Machine)

- DIR 1.1:Void pointer initialised with pointer to an instance of structure LinSM_InternalConfigType, No memory allignment issues expected. Warning supressed intentionally.
- RULE 11.5: Warning supressed intentionally- the pointer being typecasted is of the matching type, No memory allignment issues expected

2.15 RTA-MEM

2.15.1 Ea (EEPROM Abstraction)

No violations

2.15.2 Fee (Flash EEPROM Emulation)

- DIR 1.1: Warning for variants where Fee_Prv_FsIfGetLogicalRowSize returns FEE_PRV_MAX_UINT32; these never enter the enclosing else-if branch.
- RULE 10.5: Cast is uncritical, because i_u32 will not exceed FEE_RB_USER_MAX_E
- RULE 10.5: Cast is uncritical, because i_u8 will not exceed FEE_RB_JOBTYPE_MAX_E
- RULE 11.5: uint8 alignment is not stricter than void alignment
- RULE 11.6: Addresses of type uint32 can safely be casted to pointer in this

situation as it guaranteed that the code is activated if the pointer size is <=4bytes

- RULE 8.13: AR4.2 defines this function prototype with a non-constant pointer
- RULE 8.13: DataBufferPtr is not const because it is also not const in the public header file

2.15.3 MemAcc

- DIR 1.1: Memory size does not matter as this part is only used on a PC which has suitable memory → no issue
- RULE 10.5: The cast of 0 to an enum is unproblematic as 0 is always available in the enum due to the character of the enum generation → OK
- RULE 10.5: type cast to enum is not causing wrong value, as the stored values are the same as the enum-type ones
- RULE 14.3: looping one time or more time is intended and depends on config → OK
- RULE 2.4: Type definition is in use - wrong message → code is ok
- RULE 8.13: DeviceData_pst is changeable in some configurations → OK
- RULE 8.13: JobData_pst is changeable in some configurations → OK
- RULE 8.13: pointer parameters are changeable in some configurations → OK
- RULE 8.4: actually this is static but made global so that it can be measured/no external access → no risk
- RULE 8.5: multiple declarations of these objects might occur in the project as long as the definition fits, no risk

2.15.4 MemIf (Memory Abstraction Interface)

- RULE 5.9: declaration/definition of LOCAL_INLINE functions are split on different headers for hiding implementation - no issue

2.15.5 NvM (NVRAM Manager)

- DIR 1.1 RULE 11.5, 11.6: Casting to byte pointer can always be done safely
- DIR 1.1, 11.6, RULE 11.5: Casting to byte pointer can always be done safely
- DIR 1.1, RULE 1.1, 8.6, 14.3: RB compiler assert is using mechanisms which makes the violations necessary.
- DIR 1.1, RULE 11.6: Casting to void pointer can always be done safely
- DIR 4.1, RULE 1.3, 18.1: The caller of this function makes sure that all pointers are valid
- RULE 1.3, 18.1 DIR 4.1: Index is always in bounds, Pointer value is always valid per configuration.
- RULE 10.1: This function ensures that enum type is used only in valid range

- RULE 10.3: this enum is always in uint8 range and can be safely converted to uint8
- RULE 10.5: The value range has been ensured to be within the permissible values of the MemIf_JobResultType enumeration
- RULE 10.5: conversion to enum value range is ok, because source contains only values of this type
- RULE 13.5: Skipped functions are only getter and have no side-effect
- RULE 13.5: The side effect that right-side function is not called if idxDevice_uo is out-of-range is intended and necessary
- RULE 13.5: both function are pure getter, no unexpected side-effects
- RULE 13.5: if Nv-CRC is configured, then Nv-CRC shall be checked, no unexpected side-effects
- RULE 13.5: if function pointer is defined, the function is called, no unexpected side-effects
- RULE 13.5: no hidden side effects as called functions do not modify RAM
- RULE 13.5: the function is called only if block ID is valid, no unexpected side-effects
- RULE 13.5: the function is called only if queue ID has required value, no unexpected side-effects
- RULE 13.5: the function is called only if required service is active, no unexpected side-effects
- RULE 13.5: the function is called only if required services are pending, no unexpected side-effects
- RULE 16.4: Default can not be reached and has no effect here
- RULE 20.7: To create the list the argument shall be not parenthesized
- RULE 5.9, 8.13: To simplify unit test the declaration and definition of this inline function is located in two different header files. The statements within the function can be disabled via a compiler-switch but the parameter is still required to be changeable if the statements within the function are active.
- RULE 5.9: To simplify unit test the declaration and definition of this inline function is located in two different header files.
- RULE 8.13: All CheckBlockData functions shall have the same type
- RULE 8.13: All IsBlockParticipating functions shall have the same signature and in IsBlockParticipating functions where an error can occur Errors_puo has to be changeable
- RULE 8.13: In case the block header feature is deactivated the function still exists with no code inside. In this case the pointer could be of type 'pointer to const'. That the buffer is not modified is on purpose and does not create a problem here.
- RULE 8.13: Prototype of function cannot be changed as a common type is used → using the const has no impact

- RULE 8.13: The statements within the function can be disabled via a compiler-switch but the parameter is still required to be changeable if the statements within the function are active.
- RULE 8.13: This function does not change the job-structure but all job-resource-cancel-functions shall have the same signature and especially the job-structure has to be changeable so that the job-result can be modified if necessary.
- RULE 8.13: This function does not update the result structure but all Job-State-functions shall have the same signature and especially the result-structure has to be changeable so that the result can be modified if necessary.
- RULE 8.13: This message is correct only for some configuration variants, it has no functional impact → OK
- RULE 8.5, 8.6, 8.11 DIR 1.1: If we declared this in NvM_Cfg.h, we would effectively re-export this to all NvM users. Either use short identifier or configure the compiler to differentiate more than 63 chars.
- RULE 8.5, 8.6: Not using a header file for this declaration is a customer requirement
- RULE 8.5: Multiple declaration of this function is due to the fact that RTE also generates a header file.
- RULE 8.5: Not using a header file for this declaration is a customer requirement

2.15.6 rba_EaFs2

- RULE 11.3: API takes only this pointer
- RULE 8.13: DataBufferPtr is not const because of backward compatability issues

2.15.7 rba_FeeFs1x

- DIR 16.4: default statement should never be reached
- RULE 10.5: casting is safe here because number of block copies are limited to 9
- RULE 11.3: The accessed addresses in DFLASH and in the Medium buffer always are 32Bit aligned
- RULE 11.3: uint8 can always be converted safely to void

2.15.8 rba_FeeFs2

- RULE 10.5: Casting the uint8 to an enum enables compact implementation
- RULE 11.3: Function Fee_Prv_MediumNext expects a pointer of type: void const
- RULE 11.5, 18.6: Alignment of pointer is assured by Fee_Prv_MediumPre

function, returns the pointer to a global buffer

- RULE 18.6: Returns the pointer to a global buffer
- RULE 8.13: The pointer addressed by the pointer parameter 'marker_ppcst' is modified in Fee_Prv_MediumNext. It cannot be of type 'pointer to const'

2.15.9 rba_FeeFs3

- RULE 11.4: Cast required to access block headers in medium buffer via pointer; 32-bit alignment of header is enforced and special integer rba_MemLib_UintPtr_tuo is used.
- RULE 11.4: Cast required to access block headers in uint32 buffer via pointer; 32-bit alignment of header is enforced and special integer rba_MemLib_UintPtr_tuo is used.
- RULE 11.4: Casts required if block header doesn't fill a complete virtual page; 32-bit alignment of header is enforced and special integer rba_MemLib_UintPtr_tuo is used.
- RULE 5.9: Declarations and definitions of LOCAL_INLINE functions are split on different headers to hide implementation - no issue

2.15.10 rba_MemLib

- RULE 1.1: static checker tools is not capable to correctly evaluate this target specific check - no issue as this is a compile-time check
- RULE 1.2,20.13 Dir 1.1: warning pre-processor comment only available for some compilers
- RULE 10.3: A non-constant expression is assigned to unsigned int - no issue (configuration dependent)
- RULE 10.3: A non-constant expression is assigned to unsigned int - no issue (configuration dependent)
- RULE 11.1: Casts to uint32 needed to get a part of function address value - not critical
- RULE 11.4: Cast of Pointers via integer is necessary as following access is performed with 32bit alignments. Alignment was ensured before. Conversion uses special integer type rba_MemLib_UintPtr_tuo
- RULE 11.4: Casts to uint32 needed to perform following alignment checks on the addresses - not critical
- RULE 11.4: Pointer to/from integer is necessary as following access is performed with 32bit alignments. That was ensured before → not critical
- RULE 11.4: Pointer to/from integer is necessary as following access is performed with 32bit alignments. That was ensured before → not critical as rba_MemLib_UintPtr_tuo is used as temporary integer
- RULE 5.9: declaration/definition of LOCAL_INLINE functions are split on different headers for hiding implementation - no issue

- RULE 5.9: declaration/definition of RBA_MEMLIB_LOCAL_INLINE functions are split on different headers for hiding implementation - no issue

2.15.11 rba_MemWrprMemAcc

No violations

2.16 RTA-RipsPlugins

2.16.1 Rte_Rips (RTE Implementation Plugin Support)

- RULE 11.3: The warning is non-removable since casting is required to manipulate the data as uint16/uint32 type. The warning is uncritical since the data size is reduced and therefore the alignment is even better than required. Approved by MISRA2012_SUPPORT_TEAM.
- RULE 11.3: The warning is non-removable since casting is required to manipulate the data as uint8/uint16 type. The warning is uncritical since the data size is reduced and therefore the alignment is even better than required. Approved by MISRA2012_SUPPORT_TEAM.
- RULE 11.4,11.6: The warning is uncritical and non-removable since casting is required to manipulate memory address as uint32. Approved by MISRA2012_SUPPORT_TEAM.
- RULE 11.5: The warning is non-removable since pointer to void is required to have same syntax as C90 interface and casting is required to manipulate the data. The warning is uncritical since the address range and proper alignment is ensured by the calculation done before in integer representation. Approved by MISRA2012_SUPPORT_TEAM.

2.16.2 Rte_Rips_CSSafety

- DIR 1.1: Cast from a pointer to object type to a pointer to void is necessary for CallTrustedFunction syntax, Autosar guidelines
- DIR 1.1: Cast from a pointer to object type to a pointer to void is necessary for Memcopy function.
- DIR 1.1: Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters.
- RULE 11.3, 12.3:Casting to different object pointer type is due to difference in IOC and RTE. Function-like macros may have to use the comma operator. Function-like macros are required for efficiency reasons [SRS_BSW_00330].
- RULE 11.3, 12.3:Casting to different object pointer type is due to difference in IOC and RTE.Function-like macros may have to use the comma operator. Function-like macros are required for efficiency reasons [SRS_BSW_00330].
- RULE 11.5, DIR 1.1: Casting “FunctionParams” is required since CallTrustedfunction parameter is of void pointer type.Cast from a pointer to object type

to a pointer to void is necessary for Memcopy function.

- RULE 11.5, DIR 1.1:Casting “FunctionParams” is required since CallTrusted-function parameter is of void pointer type.Cast from a pointer to object type to a pointer to void is necessary for Memcopy function.
- RULE 11.5: This casting is required since CallTrusted function parameter is of void pointer type.
- RULE 12.3, 20.7: Warning is due to optimization of API by IOC
- RULE 12.3: Function-like macros may have to use the comma operator. Function-like macros are required for efficiency reasons [SRS_BSW_00330].
- RULE 16.4: Empty default case is intended here
- RULE 17.7: No need to evaluate the return value
- RULE 8.13, 8.2, DIR 1.1: Function syntax based on AUTOSAR_SWS_RTE 7.2.4.9.1 so parameters cannot be const, Function might have empty parameters depending on the configuration,Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters.
- RULE 8.13: Function syntax is according to AUTOSAR_SWS_OS 8.3.5
- RULE 8.2: Function might have empty parameters depending on the configuration.
- RULE 8.4, DIR 1.1: Below variable is not used any where else outside the file so external linkage is not present and not required any previous declaration.Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters.
- RULE 8.5: Below function will be used by external module hence extern declaration is provided

2.16.3 Rte_Rips_CSXfrm (RTE Implementation Plugin Support for C/S Communication with Transformers)

- DIR 1.1: Array is a constant pointer, Hence it is not required to pass address.
- DIR 1.1: Cast from a pointer to object type to a pointer to void is necessary for Com_ReceiveDynSignal.
- DIR 1.1: Cast from a pointer to object type to a pointer to void is necessary for Com_ReceiveSignal.
- DIR 1.1: Cast from a pointer to object type to a pointer to void is necessary for Com_SendDynSignal.
- DIR 1.1: Cast from a pointer to object type to a pointer to void is necessary for Com_SendSignal.
- DIR 1.1: Cast from a pointer to object type to a pointer to void is necessary for Memcopy function.
- DIR 1.1: Cast from a pointer to object type to a pointer to void necessary, as API uses void pointer as described by AUTOSAR
- DIR 1.1: Since the identifier name is configuration dependent, we cannot

assure the uniqueness of first 63 characters.

- RULE 10.3: If the PduLengthType is configured as unsigned type, 'unsigned short', the project has to ensure that the length value is not exceeding 16 bit
- RULE 11.8: Pointer cast is required when data element is of array/Structure type
- RULE 13.5: The State Variable will be updated with RTE_RIPS_REQ_POSSIBLE or RTE_RIPS_REQ_RESP_READ hence this condition will satisfy either case
- RULE 15.4: In case of N:1 server based on signal code is generated. Hence multiple break is availble,
- RULE 17.7: Requirement is not provided in specification for return value,So how to handle return value is not clear. Though code is verified manually there is no impact on functionality
- RULE 2.7, DIR 1.1: RULE 2.7- API is Generated as per requirement. DIR1.1-Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters.
- RULE 2.7, DIR 1.1: Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters. 2.7- API Generation is as per requirement
- RULE 2.7: API Generation is as per requirement
- RULE 2.7: API is Generated as per requirement.
- RULE 20.7: Simple macro argument expression is not parenthesized
- RULE 5.2, 5.1, DIR 1.1: Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters.
- RULE 8.13, DIR 1.1: RULE 8.13 The function is generated to be compliant with RTE call function. Hence Pointer to var is used instead of Pointer to const. DIR 1.1- Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters.
- RULE 8.4, 2.7, DIR 1.1: Below function is not used any where else outside the file so external linkage is not present and not required any previous declaration. 2.7 - API is generated as per Requirement. DIR1.1-Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters.
- RULE 8.4, 2.7: Below function is not used any where else outside the file so external linkage is not present and not required any previous declaration. 2.7 - API is generated as per Requirement
- RULE 8.4, 5.1, 5.2 DIR 1.1: Below variable is not used any where else outside the file so external linkage is not present and not required any previous declaration.Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters.
- RULE 8.4, 5.1, 5.2, DIR 1.1: Below variable is not used any where else outside the file so external linkage is not present and not required any previous declaration. Since the identifier name is configuration dependent, we

cannot assure the uniqueness of first 63 characters.

- RULE 8.4, 5.1, 5.2, DIR 1.1: Below variable is not used any where else outside the file so external linkage is not present and not required any previous declaration.Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters.
- RULE 8.4, DIR 1.1: Below variable if LdComTp, is not used any where else outside the file so external linkage is not present and not required any previous declaration. DIR 1.1- Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters.
- RULE 8.4, DIR 1.1: Below variable is not used any where else outside the file so external linkage is not present and not required any previous declaration.
- RULE 8.4, DIR 1.1: Below variable is not used any where else outside the file so external linkage is not present and not required any previous declaration. DIR 1.1- Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters.
- RULE 8.4, DIR 1.1: Below variable is not used any where else outside the file so external linkage is not present and not required any previous declaration.Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters.
- RULE 8.4, DIR 1.1: RULE 8.4- Below variable is not used any where else outside the file so external linkage is not present and not required any previous declaration. DIR 1.1- Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters.
- RULE 8.4,5.1,5.2 DIR 1.1: Below variable is not used any where else outside the file so external linkage is not present and not required any previous declaration.Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters.
- RULE 8.4,5.1,5.2 DIR 1.1: Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters.
- RULE 8.4,5.1,5.2, DIR 1.1: Below variable if LdComTp, is not used any where else outside the file so external linkage is not present and not required any previous declaration. RULE 5.1,5.2 DIR 1.1- Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters.
- RULE 8.4,5.1,5.2, DIR 1.1: Below variable if LdComTp, is not used any where else outside the file so external linkage is not present and not required any previous declaration. RULE 5.1,5.2DIR 1.1- Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters.
- RULE 8.4: Below boolean array variable is not used any where else outside the file so external linkage is not present and not required any previous declaration.
- RULE 8.4: Below function implementation is user dependent and will be called by user so external
- RULE 8.4: Below function in not used externaly, so external linkage not found hence declaration not required

- RULE 8.4: Below function is not used any where else outside the file so external linkage is not present and not required any previous declaration.
- RULE 8.4:Below code is not used any where else outside the file so external linkage is not present and not required any previous declaration.
- RULE 8.5, 8.4 DIR 1.1: Since identifier is used in multiple file. Below variable is not used any where else outside the file so external linkage is not present and not required any previous declaration.Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters.
- RULE 8.5, DIR 1.1: RULE 8.5 - Below API is used in multiple file, hence declared as extern as other place. DIR 1.1- Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters.
- RULE 8.5, DIR 1.1: RULE 8.5 - Below API is used in multiple file, hence declared as extern as other place. DIR 1.1-Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters.
- RULE 8.5, DIR 1.1: RULE 8.5 - Below function will be used by external module hence extern declaration is provided in other file. DIR 1.1- Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters.
- RULE 8.5, DIR 1.1: RULE 8.5 - Below function will be used by external module hence extern declaration is provided.
- RULE 8.5, DIR 1.1: RULE 8.5 - Below function will be used by external module hence extern declaration is provided. DIR 1.1- Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters.
- RULE 8.5, DIR 1.1: RULE 8.5-Below function will be used by external module hence extern declaration is provided DIR 1.1-Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters.
- RULE 8.5: Below function will be used by external module hence extern declaration is provided
- Rule 1.3,18.1: Here object is not a pointer type, Hence accessing element of structure is accessed via . operator, which will not cause any dereference issue
- Rule 8.13: API Prototype is as per requirement

2.16.4 Rte_Rips_Sig2Srv

- DIR 1.1: Cast from a pointer to object type to a pointer to void is necessary for Com_SendSignal.
- DIR 1.1: Cast from a pointer to object type to a pointer to void is necessary for Com_SendSignalGroupArray.
- DIR 1.1: Cast from a pointer to object type to a pointer to void is necessary for Memcopy function.

- DIR 1.1: Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters.
- RULE 1.1, 10.3: Conversion from signed type to unsigned char is necessary to evaluate the return type from the Inverse Transformers.
- RULE 1.1, 10.3: Conversion from signed type to unsigned char is necessaryto evaluate the return type from the Inverse Transformers.
- RULE 2.7: Usage of info inside the Api is still unclear and same clarification is requested from Module
- RULE 8.4, DIR 1.1: Below variable is not used any where else outside the file so external linkage is not present and not required any previous declaration.Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters.
- RULE 8.5, DIR 1.1: RULE 8.5 - Below function will be used by external module hence extern declaration is provided. DIR 1.1- Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters.
- RULE 8.5: Below function will be used by external module hence extern declaration is provided
- RULE 8.6: external linkage is declared but is defined externally

2.16.5 Rte_Rips_SRCom

- DIR 1.1: Cast from a pointer to object type to a pointer to void is necessary for Com_ReceiveDynSignal.
- DIR 1.1: Cast from a pointer to object type to a pointer to void is necessary for Com_ReceiveSignal.
- DIR 1.1: Cast from a pointer to object type to a pointer to void is necessary for Com_SendDynSignal.
- DIR 1.1: Cast from a pointer to object type to a pointer to void is necessary for Com_SendSignal.
- DIR 1.1: Cast from a pointer to object type to a pointer to void is necessary for Memcopy function.
- DIR 1.1: Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters.
- RULE 1.1, 10.3: Conversion from signed type to unsigned char is necessary to evaluate the return type from the Inverse Transformers.
- RULE 1.1, 10.3: Conversion from signed type to unsigned char is necessaryto evaluate the return type from the Inverse Transformers.
- RULE 17.7: Requirement is not provided in specification for return value,So how to handle return value is not clear. Though code is verified manually there is no impact on functionality
- RULE 2.7: Usage of info inside the Api is still unclear and same clarification is requested from Module

- RULE 8.13: API Prototype is as per requirement
- RULE 8.4, DIR 1.1: Below variable is not used any where else outside the file so external linkage is not present and not required any previous declaration.
- RULE 8.4, DIR 1.1: Below variable is not used any where else outside the file so external linkage is not present and not required any previous declaration.Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters.
- RULE 8.4: Below function in not used externaly, so external linkage not found hence declaration not required
- RULE 8.4:Below code is not used any where else outside the file so external linkage is not present and not required any previous declaration.
- RULE 8.5, DIR 1.1: RULE 8.5 - Below function will be used by external module hence extern declaration is provided.
- RULE 8.5, DIR 1.1: RULE 8.5 - Below function will be used by external module hence extern declaration is provided. DIR 1.1- Since the identifier name is configuration dependent, we cannot assure the uniqueness of first 63 characters.
- RULE 8.5: Below function will be used by external module hence extern declaration is provided
- RULE 8.6: external linkage is declared but is defined externally
- Rule 11.8,20.7: Expression Shall not cause any dereference issue

2.17 RTA-SAFE

2.17.1 E2E (End to End Library)

- RULE 2.2: This operation is not redundant
- Rule 10.1 Violation: The expression evaluation doesnot compare to the boolean value. Checks the datalength value is a byte value or not. No functional impact is observed
- Rule 10.3,10.4 Violation: Both sides of the expression are of type uint16. Does not lead to any functional issue
- Rule 14.2 Violation: Loop control variable in this 'for' statement, index, is modified in the body of the loop. Does not lead to any functional issue
- Rule 14.4 Violation: Controlling expression is not an 'essentially Boolean' expression. Does not lead to any functional issue
- Rule 15.7 Violation: No concluding 'else' exists in this 'if'-'else'-'if' statement. Does not lead to any functional issue
- Rule 16.4 Violation: CounterLength can only be 2, 4 or 8, hence default case is not relevant
- Rule 20.7 Violation: Simple macro argument expression is not parenthesized. Does not lead to any functional issue

2.17.2 E2EXf (End to End Library Transformer)

- RULE 1.3: NULL pointer validation accepted as per AR specification.
- RULE 10.3: Counter datatype depends on E2E profiles. No functional impact
- RULE 8.13: API defined in AR specification.
- RULE 8.4: The object is only referenced in the current file. Accepted for better generated code structure
- RULE 8.7: The object is only referenced in the translation unit where it is defined. Accepted for better generated code structure

2.17.3 WdgIf (Watchdog Interface)

- DIR 4.10: This file will not be included as there is no PostBuild configurations enabled

2.17.4 WdgM (Watchdog Manager)

- DIR 4.1, RULE 1.3, 11.4: Function below is needed because C99 _Alignof() is not available in our enviroment There is no real NULL-pointer dereference.
- RULE 11.4: Safe, uintptr_t is used.
- RULE 11.4: uintptr_t is used to guarantee there is no information loss
- RULE 11.5, 11.6: Guaranteed sane casting
- RULE 11.5: Guaranteed sane casting
- RULE 11.5: Sane casting, guaranteed by the previous operations
- RULE 15.5: Multiple 'return' paths are used to handle the return value in different error scenarios. Thus, the warning can be ignored
- RULE 15.7: This is added to fix MISRA warning. This is fine.
- RULE 18.7: This is fine.
- RULE 2.7: The variable "Fifo_CaptureTime" is used inside function with conditional Macros. This is fine to be used.
- RULE 20.5: Macro WDGM_PRV_AVOID_RTE_PROVIDED_FUNC_DECLARATION is undefined to access WdgM_MainFunction, WdgM_GetLocalStatus, WdgM_GetGlobalStatus, WdgM_GetMode, WdgM_PerformReset, WdgM_SetMode, WdgM_GetFirstExpiredSEID, and WdgM_CheckpointReached Api's for dependency module, thus the warning canbe ignored
- Rule 11.5: QAC problem… void → uint8 is always safe
- Rule 18.3: QAC problem…

2.18 RTA-SEC

2.18.1 CryIf (Crypto Interface)

- RULE 11.8: const converted to volatile because of AR spec. ,but it will be only read
- RULE 8.13 : keyStatusPtr can not be const because this interface is defined by AUTOSAR
- RULE 8.13 : resultPtr and resultLengthPtr can not be const because this interface is defined by AUTOSAR
- RULE 8.13: job can not be const because this interface is defined by AUTOSAR
- RULE 8.13: parameter can not be const because this interface is defined by AUTOSAR
- RULE 8.13: pointer parameters can not be const because this interface is defined by AUTOSAR
- RULE 8.13: pointer parameters cannot be const because this interface is defined by AUTOSAR
- RULE 8.13: the pointer can not be const because this interface is defined by AUTOSAR
- RULE 8.13: verifyPtr can not be const because this interface is defined by AUTOSAR
- RULE 8.5, 8.3: If expected apis are routed to the same function, then it may happen that the same function will be declared multiple times. The declarations might even have different parameter names (but not types). This should have no negative side effect.
- RULE 8.5: If expected apis are routed to the same function, then it may happen that the same function will be declared multiple times.

2.18.2 Crypto

- DIR 4.1 RULE 1.3: This function is only ever called to remove a job from a queue after dispatching or canceling it, therefore the corresponding pointer will always be valid.
- DIR 4.1 RULE 1.3: This function is quite deep into the job processing logic, neither the driver object pointer nor the job pointer can be invalid here.
- DIR 4.1 RULE 1.3: We can only ever get here if the queueing logic pulled a valid job pointer from the queue stack.
- RULE 10.5: Casting between enums is safe as both are 8-bit with matching value ranges and semantically equivalent definitions.
- RULE 11.3: Casting from stricter alignment back to a less strict alignment
- RULE 11.3:Casting is necesarry for variable array lengths and will not impact

on any functionality

- RULE 11.4, 11.6: Casting pointer to uint8 is needed because of insufficient C90 standard
- RULE 11.4, 11.6: Casting void pointer to check alignment
- RULE 11.5: Casting pointer to bigger type for efficient memcopy
- RULE 11.5: Casting to uint8 pointer for byte to byte copy
- RULE 11.5: Casting to uint8 pointer for byte to byte memset
- RULE 11.5: Casting volatile uint16 pointer type to do simple memory manipulation
- RULE 11.5: Casting volatile uint32 pointer type to do simple memory manipulation
- RULE 11.8 DIR 1.1: Loss of const information is necessary. Input parameters are declared as (void) for generic implementation
- RULE 11.8: Based on the configuration the jobQueue_pst sometimes points to a const or volatile structure, the function using it will handle the object accordingly
- RULE 14.3: Condition is always false when no key is configured.
- RULE 14.3: Condition is always true when no key is configured.
- RULE 18.6: False positive the keyElement size ptr has greater lifetime
- RULE 2.7: Parameter cryptoKeyId_u32 may not be used when no key is configured.
- RULE 2.7: Parameter key_pcst and keyTarget_pcst may not used when no key is configured.
- RULE 2.7: Parameter key_pcst may not be used when no key is configured.
- RULE 2.8: If a key element is orphaned (i.e. not referenced by any key), this structure will not be used. Our validator throws a warning for such cases, so the integrator will be aware.
- RULE 8.13: In a minimal config without keys, the parameters are unused and could therefore be constant. This does not apply for regular configurations with at least one key.
- RULE 8.13: Parameter jobExt_pst may not modified when no key is configured.
- RULE 8.13: Parameter job_pst may not get modified when no keys are configured.
- RULE 8.13: Parameter types cannot be changed because a common function pointer type used to reference the function
- RULE 8.13: Parameter types cannot be changed because this public function is defined by AUTOSAR, to have this parameter volatile has no negative effect
- RULE 8.13: the pointers can not be const because this interface is defined by

AUTOSAR

2.18.3 Csm (Crypto Service Manager)

- DIR 1.1, RULE 8.5: Multiple identifier within the same name space cannot be avoided because of AR definition. This function must be declared here before its definition, but it is also redeclared in 'Rte_Csm.h'. This should have no negative side effect.
- DIR 1.1: Larger than 32767 bytes object is not causing any issues
- RULE 10.5: Casting between enums is safe as both are 8-bit with matching value ranges and semantically equivalent definitions.
- RULE 11.3:Casting is necesarry for variable array lengths and will not impact on any functionality
- RULE 11.4, 11.6: Cast from pointer type to integral type is necessary in this case. The Ptr value of the task/interrupt structure is compared because the OS always delivers the same ptr value for the same task/interrupt. In addition, task/interrupt structure addresses are always unique and thus represent a robust criterion for comparison.
- RULE 11.5,11.8: cast cannot be avoided because of AUTOSAR definition
- RULE 11.5,11.8: cast cannot be avoided because of AUTOSAR definition [SWS_Csm_Rb_91039]
- RULE 11.5,11.8: cast cannot be avoided because of AUTOSAR definition [SWS_Csm_Rb_91060]
- RULE 11.5: cast cannot be avoided because of AUTOSAR definition [SWS_Csm_91036],[SWS_Csm_91028]
- RULE 11.8: cast cannot be avoided because of AUTOSAR definition
- RULE 13.5: some compilers does not support #pragma PRQA_NO_SIDE_EFFECTS (for Csm_Prv_IsParamOK)
- RULE 14.3: The for loop might not be enter in case of CSM_CFG_-FIRST_NONLEGACY_PARTITION_INDEX and CSM_CFG_PARTITION_COUNT are equal
- RULE 8.13: job can not be const because this interface is defined by AUTOSAR
- RULE 8.13: pointer parameters cannot be const because this interface is defined by AUTOSAR
- RULE 8.13: tagPtr, tagLengthPtr can not be const because this interface is defined by AUTOSAR
- RULE 8.13: the type 'pointer to const' cannot be avoided because of AUTOSAR definition
- RULE 8.13: verifyPtr can not be const because this interface is defined by AUTOSAR
- RULE 8.5, 8.3: This function must be declared here before its definition, but it

is also redeclared in 'Rte_Csm.h'. This should have no negative side effect. The parameter naming inconsistencies are tracked and will be synchronized soon.

- RULE 8.5: It is highly likely that the callback function is redeclared with external linkage in a different unit, which also owns its definition. This should have no negative side effect.
- RULE 8.5: Multiple declarations of external object or function. Second declarations are coming from RTE interface for internal usage. No negative effect.

2.18.4 IdsM

- RULE 1.3, 18.1, Dir 4.1: It is ensured that the pointer values are correct by calling functions
- RULE 11.3: Casting from stricter alignment back to a less strict alignment
- RULE 11.4, 11.6: Casting void pointer to check alignment
- RULE 11.5: Casting pointer to bigger type for efficient memcopy
- RULE 11.5: Casting to uint8 pointer for byte to byte copy
- RULE 11.5: Casting to uint8 pointer for byte to byte memset
- RULE 11.5: Casting volatile uint16 pointer type to do simple memory manipulation
- RULE 11.5: Casting volatile uint32 pointer type to do simple memory manipulation
- RULE 14.3: finding least severe Event index is only applicable if IDSM_CFG_NUM_EVENT_BUFFERS is greater than 1
- RULE 14.3: finding least severe QSEv index is only applicable if IDSM_CFG_NUM_QSEVBUFFERS is greater than 1
- RULE 2.8: This ROM block data is referenced inside NvM module under Array of block descriptors
- RULE 8.13: Sampling filter and custom filter can modify IdsM_EventBuffer_tst hence array of function pointer created with (IdsM_EventBuffer_tst) instead of (const IdsM_EventBuffer_tst)
- RULE 8.13: contextDataBufferSize_u16 and bufferIndex_u16 are out variables which are modified in when context data buffers are configured.
- RULE 8.13: contextData_pcau8 is a out variable which is modified in when context data buffers are configured.
- RULE 8.13: countContextData is inout variables which is modified in when context data buffers are configured.
- RULE 8.13: eventBuffer Cannot be const pointer to keep the filter interfaces generic
- RULE 8.13: eventBuffer2_pst is inout variables which is modified in when context data buffers are configured.
- RULE 8.13: eventBuffer_pst is inout variable whose value is modified in when

context data buffers are configured.

- RULE 8.13: eventBuffer_pst is inout variables which is modified in when context data buffers are configured.
- RULE 8.13: writableIndex_pu16 is out variables which is modified in when context data buffers are configured.
- RULE 8.3:Two declarations exists for this api. One is in IdsM_Cbk.h & another generated by Dcm. Hence,the difference in identifiers.
- RULE 8.5, 8.3: Two declarations exists for this api. One is in IdsM_ReportSecurityEventWrapper.c & another generated by Rte. Hence,the difference in parameters.
- RULE 8.5: the function declaration is also generated by the CSM, this does not cause any issues
- RULE 8.5: the function declaration is also generated by the Dcm, this does not cause any issues
- RULE 8.5: the function declaration is also generated by the Rte, this does not cause any issues
- RULE 8.7:

2.18.5 KeyM

- DIR 1.1: Casting to void pointer is for efficient memory copy
- RULE 11.5: Casting to uint16 pointer is for efficient memory copy
- RULE 11.5: Casting to uint32 pointer is for efficient memory copy
- RULE 11.6: Casting to uint32 is to check alignment
- RULE 8.5: The same function is redeclared with external linkage in Csm. This declaration should have no negative side effect.

2.18.6 rba_CryptoAuCSC

- RULE 10.5: The uint8 value is transferred directly from the EcuC where only values that directly match a cycursoc_CryptoEngine value are permitted.
- RULE 11.4, 11.3: Cast necessary between CycurSoC data pointer and AUTOSAR data pointer
- RULE 11.4, 11.3: Cast necessary between CycurSoC data pointer and AUTOSAR data pointer (alignment will be always valid)
- RULE 11.4, 11.3: Cast necessary between CycurSoC length pointer and AUTOSAR length pointer (alignment will always be valid)
- RULE 11.4, 11.3: cast necessary between cycurSoc data pointer and AUTOSAR data pointer (alignment will be always valid)
- RULE 14.3: Condition is always false when no key is configured.
- RULE 2.8: This array will only be unused if there is not a single driver object configured for this Crypto driver.

- RULE 8.13: Parameter types cannot be changed because a common function pointer type used to reference the function
- RULE 8.5: We need to declare this public API to enable direct requests. Similarly, CryIf might need and therefore declare it as well. This can result in multiple declarations of the same function. Such a scenario should not have any negative side effects.

2.18.7 rba_CryptoAuHSM3

- RULE 10.5: Cast needed because the Csai API is explicitly intended to be used with key handles by casting them to the ecy_hsm_Csai_KeyIdT type.
- RULE 11.3, 11.5: Cast needed to update provide the tag tree in uint32 format, which was already prepared to be cast from uint8.
- RULE 11.8,10.4: Cast needed to update the job input parameter and the cast to uint8 is safe because the origin of input pointer is a uint8 .
- RULE 11.8: Cast needed to link incompatible interfaces and to avoid const warning
- RULE 8.13: If no key configured, certificateKey_pcst and currentKeyElementTagTree_ppct are not used but the pointers could not be of type 'pointer to const'.
- RULE 8.13: Parameter types cannot be changed because a common function pointer type used to reference the function
- RULE 8.13: Parameter types cannot be changed because job pointer is only not modified if no buffers are configured
- RULE 8.13: pointers can not be const because a common function pointer type used to reference the function
- RULE 8.5: We need to declare this public API to enable direct requests. Similarly, CryIf might need and therefore declare it as well. This can result in multiple declarations of the same function. Such a scenario should not have any negative side effects.

2.18.8 rba_CryptoCCL

- DIR 1.1: Larger than 32767 bytes object is not causing any issues
- DIR 1.1: Object might exceed 32767 bytes due to high memory consumption of internal CycurLib context structures that are stored inside.
- DIR 1.1: input parameters are declared as (void) for generic implementation
- DIR 1.1: input parameters are declared as (void) for generic implementation.
- DIR 1.1: void pointer must be used because of limitations of C89 standard
- RULE 10.5, 10.8: Cast needed to ensure compatibility between job redirection configuration and redirection configuration type
- RULE 10.5: cast is required for time-independent behaviour. C-standard: operator shall yield 0 or 1

- RULE 11.4, 11.6: Cast from pointer type to integral type is necessary in this case. The Ptr value of the task/interrupt structure is compared because the OS always delivers the same ptr value for the same task/interrupt. In addition, task/interrupt structure addresses are always unique and thus represent a robust criterion for comparison.
- RULE 11.4, 11.6: Cast necessary and alignment checked by rba_CryptoCCL_Prv_Lib_Core_Au8Lib_IsAu32Aligned above
- RULE 11.8 DIR 1.1: Loss of const information is necessary. Input parameters are declared as (void) for generic implementation
- RULE 11.8 DIR 1.1: Loss of const information is necessary. input parameters are declared as (void) for generic implementation
- RULE 11.8 DIR 1.1: input parameters are declared as (void) for generic implementation
- RULE 13.5: CheckJob function is identified has having side effects. Can be ignored.
- RULE 13.5: Get function is identified as an expression causing side effect. Can be ignored.
- RULE 13.5: IsModeBitsOK function does not have any persistent side effects.
- RULE 13.5: IsModeBitsOK function is identified has having side effects. Can be ignored.
- RULE 13.5: The exhaustive evaluation of the combined decision is not required because the function has no side effect.
- RULE 5.4: Due to the naming conventions, the definition of identifiers has to be longer than the limit for the MISRA check.
- RULE 8.13: 'curveType_ppau8' & 'curveTypeLength_pu32' must be modifiable if key element is present
- RULE 8.13: The outputLength_pu32 pointer is used by CycurLib API and can not be const.
- RULE 8.13: the pointers can not be const because this interface is defined by AUTOSAR
- RULE 8.13: the pointers can not be const because this interface is defined indirectly by AUTOSAR via the upperlayer function
- RULE 8.5: This function is redeclared with external linkage by CryIf. This should have no negative side effect.
- RULE 8.7: False-Positive. This library function is declared extern and must be accessible from other modules.
- RULE 9.3: Default initializer is the same like the initializer used here

2.18.9 rba_CryptoHSM

- DIR 1.1, 11.5: Input parameters are declared as (void) for generic implementation.

- DIR 1.1, 11.5: input parameters are declared as (void) for generic implementation
- DIR 1.1: extensionCtx is declared as (void) for generic implementation
- DIR 1.1: input parameters are declared as (void) for generic implementation
- DIR 1.1: input parameters must be checked because they should be located within a special memory location
- DIR 1.1: void pointer must be used because of limitiations of C89 standard
- DIR 4.1 RULE 1.3, 18.1: The dereferenced pointer has valid value.
- DIR 4.1 RULE 1.3, 18.1: objectId is guaranteed by previous runtime checks and code generators to be less than the rba_CryptoHSM_Prv_DriverObjectContext_ast array length
- DIR 4.1 RULE 1.3, 18.1: objectId_u32 is guaranteed by generators to be less than rba_CryptoHSM_Prv_DriverObjectContext_ast array length
- DIR 4.1 RULE 1.3, 18.1: objectId_u32 is guaranteed by rba_CryptoHSM_-CancelJob's check to be less than the rba_CryptoHSM_Prv_DriverObject-Context_ast array length
- DIR 4.1 RULE 1.3, 18.1: objectId_u32 is guaranteed by rba_CryptoHSM_ProcessJob's check to be less than the rba_CryptoHSM_Prv_DriverObjectContext_ast array length
- DIR 4.1 RULE 1.3, 18.1: objectId_u32 is guaranteed by the checks of both rba_CryptoHSM_ProcessJob and rba_CryptoHSM_CancelJob to be less than the rba_CryptoHSM_Prv_DriverObjectContext_ast array length
- DIR 4.1 RULE 1.3: the queue pointer is always valid because the caller function derives it from an array where at least one queue is assigned to all driverObjects
- DIR 8.13: The output_pau8 abd secondaryOutput_pau8 pointer is used by HSM APIs and can not be const.
- DIR 8.13: The output_pau8 pointer is used by HSM APIs and can not be const.
- RULE 10.3,10.5,10.8: The CSAI provided macro is the official supported method to extract the SHE ERC from the CSAI error returned. It is assumed that a negative value is not returned and that it is safe to set the MSB for SHE purposes.
- RULE 10.3: Cast needed to ensure compatibility between dispatch id and service id type
- RULE 10.3: The type cannot be changed to boolean as it is defined by CSAI
- RULE 10.5, 10.8, 11.8: Casting service id to enum, Cast needed to link incompatible interfaces and to avoid const warning
- RULE 10.5, 10.8, 11.8: Casting service id to enum. The cast to uint8 is not critical because, in case of decryption, the tag buffer pointer is not changed. Required for interface compatibility.
- RULE 10.5, 10.8: Cast needed to ensure compatibility between dispatch id and algo family type

- RULE 10.5, 10.8: Cast needed to ensure compatibility between dispatch id and algo mode type
- RULE 10.5, 10.8: Cast needed to ensure compatibility between dispatch id and service type
- RULE 10.5, 10.8: Cast needed to ensure compatibility between job redirection configuration and redirection configuration type
- RULE 10.5, 10.8: Casting service id to enum is safe because the uint8 value is derived from the enum which is fixed and validated for a given configuration
- RULE 10.5, 10.8: Casting session index to enum is safe because the uint8 value is derived from the enum which is fixed and validated for a given configuration
- RULE 10.5: Cast needed because the API is explicitly intended to be used with key handles by casting them to the ecy_hsm_Csai_KeyIdT type.
- RULE 10.5: Casting service id to enum is safe because the uint32 value is derived from the enum which is fixed and validated for a given configuration
- RULE 10.5: Casting service id to enum is safe because the uint8 value is derived from the enum which is fixed and validated for a given configuration
- RULE 11.3: Casting to different object pointer type. Violation created by casting (uint8) to (ecy_hsm_Csai_VKMSChecksumT), in order to optimize resource usage. (Pointer to structure points to initial member)
- RULE 11.3: Casting to different object pointer type. Violation created by casting (uint8) to (ecy_hsm_Csai_VKMSStatusT), in order to optimize resource usage. (Pointer to structure points to initial member)
- RULE 11.3: typecasted to match the parameter Type of function definition
- RULE 11.3:Casting is necessary for variable array lengths and will not impact on any functionality
- RULE 11.4, 11.6: Cast from pointer type to integral type is necessary in this case. The Ptr value of the task/interrupt structure is compared because the OS always delivers the same ptr value for the same task/interrupt. In addition, task/interrupt structure addresses are always unique and thus represent a robust criterion for comparison.
- RULE 11.4, 11.6: Casting pointer to rba_CryptoHSM_uintptr_tuo is needed because of unsufficient C90 standard
- RULE 11.4, 11.6: Casting pointer to uint32 is needed because of unsufficient C90 standard
- RULE 11.4,11.6: Casting needed because pointer must be checked
- RULE 11.8: Cast needed to link incompatible interfaces and to avoid const warning
- RULE 11.8: Cast to non const uint8 because of HSM_VKMS interface
- RULE 11.8: Loss of const information is uncritical. Pointer is only used as dummy parameter
- RULE 11.8: The cast to uint8 is not critical because, in case of decryption, the

tag buffer pointer is not changed. Required for interface compatibility.

- RULE 11.8: the job structure cannot be const because some primitives must modify the content
- RULE 13.5: 'rba_CryptoHSM_Prv_DriverObjectQueue_IsEmptyAtFifo' is just reads and compares two internal indexes
- RULE 13.5: The exhaustive evaluation of the combined decision is not required because the function has no side effect.
- RULE 14.2: For loop control variable needs to be modified to exit the loop due to failure in parsing
- RULE 17.12: A function can also be passed without &.
- RULE 17.5: The pointer provided as function parameter cannot be specified as array with constant size, because it represents an offset in one common array.
- RULE 17.5: The pointers provided as function parameters cannot be specified as arrays with constant size, because they represent offsets in one common array. They are of the exact size described by the SHE specifications.
- RULE 18.1: Apperent invalid dereference, false positive bulkType_u32 is always in valid range
- RULE 18.1: The computing of the pointer has valid value.
- RULE 18.6: QAC throwing false positive warning
- RULE 2.8, 8.4: Variable is defined without any previous declaration, since it is used only in this file
- RULE 5.4: Due to the naming conventions, the definition of identifiers has to be longer than the limit for the MISRA check.
- RULE 8.13: 'curveType_ppau8' & 'curveTypeLength_pu32' must be modifiable if key element is present
- RULE 8.13: the 'job_cpst' field cannot be const because some custom services must modify the content
- RULE 8.13: the 'job_cpst' field cannot be const because some primitives must modify the content
- RULE 8.13: the 'job_cpst' field cannot be const because the rba_CryptoHSM_Prv_HandlePrimaryOutputAndLength function modifies the content.
- RULE 8.13: the 'job_pst' field cannot be const because one of its internal fields is changed
- RULE 8.13: the 'key_pau8' and 'keyLength_pu32' fields cannot be const, because they will be written
- RULE 8.13: the 'key_pau8' field cannot be const ,because it will be written via MemCopy operation
- RULE 8.13: the pointers can not be const because this interface is defined by

AUTOSAR

- RULE 8.13: the pointers cannot be const because some custom services must modify the content
- RULE 8.4: Variable is defined without any previous declaration, since it is used only in this file
- RULE 8.5: This function is redeclared with external linkage by CryIf. This should have no negative side effect.
- RULE 9.3: Default initializer is the same like the initializer used here

2.18.10 SecOC

- DIR 1.1 : This pointer is modified in SecOC_Prv_MemCopy
- DIR 1.1 RULE 11.3: freshnessValue is the result and can not be const
- DIR 1.1: Cast is safe, converting from uint8 to void to uint8 (inside MemCopy)
- DIR 1.1: Input parameters are declared as (void) for generic implementation
- DIR 1.1: This pointer is modified in SecOC_Prv_MemCopy
- DIR 4.1 RULE 1.3, 18.1: It is ensured that currCryptoQueueEntryIndex_u32 is valid
- DIR 4.1 RULE 1.3, 18.1: It is ensured that readIndex_vu32 and queueSize_cu32 are valid
- DIR 4.1 RULE 1.3, 18.1: partitionIndex_u32 is guaranteed by generators to be less than the SecOC_Prv_RxPdu_PartitionContextBuffer_auo array length
- DIR 4.1 RULE 1.3, 18.1: partitionIndex_u32 is guaranteed by generators to be less than the SecOC_Prv_TxPdu_PartitionContextBuffer_auo array length
- DIR 4.1 RULE 1.3: Input parameters are declared as (void) for generic implementation and it is ensured in the code generation that cryptographicPduBufferIn_pu8 is always available and not a NULL_PTR
- DIR 4.1 RULE 1.3: It is ensured that cryptographicPduState_ven is not a NULL_PTR because of a FIFO implementation and checks in the calling function
- DIR 4.1 RULE 1.3: It is ensured that txAuthenticPduCtx_pst is valid, if txSecuredPduCtx_pst is not a NULL_PTR
- RULE 1.3, 18.1, Dir 4.1: It is ensured that the pointer values are correct by calling functions
- RULE 1.3, DIR 4.1: It is ensured in the code generation that cryptographicPduBufferPosition_uo is always available and not a NULL_PTR
- RULE 1.3, DIR 4.1: It is ensured in the code generation that tpBufferPosition_puo is always available and not a NULL_PTR
- RULE 10.3: Allowed by Rule CCode_Essential_003: The initializer { 0 } may be used to initialize an aggregate (structure or array)
- RULE 11.3: Casting from stricter alignment back to a less strict alignment

- RULE 11.3: freshnessValue is the result and can not be const
- RULE 11.4, 11.6: Cast from pointer type to integral type is necessary in this case. The Ptr value of the task/interrupt structure is compared because the OS always delivers the same ptr value for the same task/interrupt. In addition, task/interrupt structure addresses are always unique and thus represent a robust criterion for comparison.
- RULE 11.4, 11.6: Casting void pointer to check alignment
- RULE 11.5: Casting pointer to bigger type for efficient memcopy
- RULE 11.5: Casting to uint8 pointer for byte to byte copy
- RULE 11.5: Casting to uint8 pointer for byte to byte memset
- RULE 11.5: Casting volatile uint16 pointer type to do simple memory manipulation
- RULE 11.5: Casting volatile uint32 pointer type to do simple memory manipulation
- RULE 13.5: SecOC_Prv_RxCheckMessageLinks checks the message linker, it is safe to not be called when overrideStatus_u8 is set to PASS or SKIP.
- RULE 15.7: Final else in if-else-if construct is always required.
- RULE 15.7: Nothing to do, empty else clause required by MISRA
- RULE 16.4: Empty default case required by coding rules
- RULE 16.4: No action is necessary
- RULE 16.4: Switch covers all enum values, empty default clause required by coding guidelines.
- RULE 16.4: Switch covers all relevant cases, empty default clause required by coding guidelines
- RULE 17.12: Preceding & operator has no effect here, pointer is only checked for NULL_PTR, function call is done in correct way
- RULE 18.1: It is ensured that function SecOC_Prv_RxGetCollQueueIdFromAuthPduId returns
- RULE 18.6: QAC throwing false positive warning
- RULE 2.2: Called function is generated by RTE generator
- RULE 2.4: Structures are used for type definitions which are used in code
- RULE 2.8: Variant is used for PBS projects
- RULE 8.13: AUTOSAR interface therefore parameter can not be set to 'pointer to const'
- RULE 8.13: Internal fields of the structure instance referenced by 'rxCollPduQueueEntry_pst' are intended to be changed in case of Cryptographic PDUs when Collection PDUs are configured.
- RULE 8.13: PduInfoPtr can not be const because this interface is defined by AUTOSAR
- RULE 8.13: Value referenced by 'pduId_pu32' is intended to be changed in

case of Cryptographic PDUs when Collection PDUs are configured.

- RULE 8.13: availableDataPtr can not be const because this interface is defined by AUTOSAR
- RULE 8.13: bufferSizePtr can not be const because this interface is defined by AUTOSAR
- RULE 8.3: Parameter overrideStatus differs from RTE definition because RTE cannot handle typedef
- RULE 8.5: The same function is declared with external linkage in RTE. This redeclaration should have no negative side effect.
- RULE 8.5: The same function is redeclared with external linkage in Csm. This redeclaration should have no negative side effect.
- RULE 8.7: Function is called in SecOC_Prv.c and in TestCd_SecOC

2.19 RTA-SWCL

2.19.1 rba_CCL

No violations

2.20 RTA-XCP

2.20.1 Xcp

- DIR 1.1: Array is not used as flexible array member. The array length is well defined.
- DIR 1.1: The input parameters of NvM_WriteBlock() are declared as (void) so we have to cast.
- DIR 1.1: The input parameters of Xcp_NvMReadBlock() are declared as (void) so we have to cast.
- DIR 1.1: The input parameters of rba_BswSrv_MemCopy() are declared as (void) for generic implementation of MemCopy.
- DIR 1.1: The input parameters of rba_BswSrv_MemSet() are declared as (void) for generic implementation of MemSet.
- RULE 1.1: There is no risk in referring the Xcp_CopyOdtEntries_16 function here, so this can be ignored
- RULE 1.1: There is no risk in referring the Xcp_CopyOdtEntries_32 function here, so this can be ignored
- RULE 1.1: There is no risk in referring the Xcp_CopyOdtEntries_8 function here, so this can be ignored
- RULE 1.1: There is no risk in referring the Xcp_CopyOdtEntries_Safe function here, so this can be ignored
- RULE 1.2: Array is not used as flexible array member. The array length is well defined. This violation is raised only when: XCP_CAN_NUMBER_OF_TX_PDUS == 1

- RULE 1.2: Array is not used as flexible array member. The array length is well defined. This violation is raised only when: XCP_COMMANDS_GET_-DAQ_INFO == STD_OFF and XCP_STORE_CLEAR_DAQ_SUPPORTED == STD_OFF and XCP_MAX_DAQRAMSECTIONS == 1
- RULE 1.2: Array is not used as flexible array member. The array length is well defined. This violation is raised only when: XCP_COMMANDS_GET_-DAQ_INFO == STD_OFF and XCP_STORE_CLEAR_DAQ_SUPPORTED == STD_OFF and XCP_SYNCHRONOUS_DATA_ACQUISITION_DAQ == STD_OFF and XCP_TL_IDS == 1
- RULE 1.2: Array is not used as flexible array member. The array length is well defined. This violation is raised only when: XCP_ETH_NUMBER_OF_RX_PDU == 1
- RULE 1.2: Array is not used as flexible array member. The array length is well defined. This violation is raised only when: XCP_FR_NUMBER_OF_TRANS-PORT_LAYERS == 1
- RULE 1.2: Array is not used as flexible array member. The array length is well defined. This violation is raised only when: XCP_FR_NUMBER_OF_TX_PDUS == 1
- RULE 1.2: Array is not used as flexible array member. The array length is well defined. This violation is raised only when: XCP_MAX_EVENT_CHANNEL == 1
- RULE 10.4, 10.8: Casting is necessary to avoid overflow
- RULE 10.4, 14.3: Rule 10.4 Casting from signed to unsigned here is not a problem as XCP_SYSTEMTIMER_TICKS > 0 Rule 14.3 Macro contains the ternary operator raising this violation, it also depends on other config-parameters, so there are situations where the compare is required, thus it is left as it is
- RULE 10.4, 14.3: Rule 10.4 Casting from signed to unsigned here is not a problem as XCP_SYSTEMTIMER_TICKS > 0 Rule 14.3 Macro contains the ternary operator raising this violation, it also depends on other config-parameters,so there are situations where the compare is required, thus it is left as it is
- RULE 10.8: cast to signed datatype is not a problem, as only 3/2 bits are multiplied with 8 → result < 127
- RULE 11.1: Cast between a pointer to function is needed (CDG-MemLay-Demand)
- RULE 11.1: Function pointer to const type is not applied in Xcp on CAN to support padding
- RULE 11.3: typecasted to PduLengthType to match the AUTOSAR API Definition
- RULE 11.3: typecasted to Tcplp_SockAddrType to match the AUTOSAR API Definition
- RULE 11.4, 11.5, 11.6: We want to have a pointer to a 2-byte-aligned address at/after the given element (address), hence casting is necessary

- RULE 11.4, 11.5, 11.6: We want to have a pointer to a 4-byte-aligned address at/after the given element (address), hence casting is necessary
- RULE 11.4, 11.5, 11.6: We want to have a pointer to a 4/8-byte-aligned address at/after the given element (address), hence casting is necessary
- RULE 11.4: Cast from XCP_ADDR_TYPE to pointer is OK as XCP_ADDR_TYPE is able to store the address (see type definition)
- RULE 11.4: Cast from XCP_ADDR_TYPE to pointer is OK as XcpAddress stores the address (and is large enough → see type definition)
- RULE 11.4: Cast from pointer to XCP_ADDR_TYPE is OK as XCP_ADDR_TYPE is able to store the address (see type definition)
- RULE 11.4: Cast from pointer to XCP_ADDR_TYPE is OK as this type is able to store the address (see type definition)
- RULE 11.4: Cast to pointer of XCP_ADDR_TYPE is OK as alignment was set correctly one line above.
- RULE 11.4: Casting between integers and pointers is necessary to interpret the address
- RULE 11.4: The cast from pointer to uint32 (address) is required as the function expects an uint32 as first argument
- RULE 11.4: rba_Dme_Write wants as first param an uint32 (address) but not a pointer, so we have to cast, which is not a problem as addresses are also uint32; the third argument needs to be a uint8-pointer, AddrPtrSrc is const - to avoid compiler-warning we cast to XCP_ADDR_TYPE first
- RULE 11.5, 11.6, DIR 1.1: Xcp_MemCopy : The calling layers ensure correct alignment; Cast to const is no problem, as the variable is only read The input parameters of rba_BswSrv_MemCopy() are declared as (void) for generic implementation of MemCopy.
- RULE 11.5, 11.6, DIR 1.1: Xcp_MemCopy : The calling layers ensure correct alignment; Cast to const is no problem, as the variable is only read The input parameters of rba_BswSrv_MemCopy() are declared as (void) for generic implementation of MemCopy.
- RULE 11.5, 18.1, 21.18 DIR 1.1, 4.1: The calling layers ensure correct alignment. Casting over void is necessary as otherwise some compilers complain Access outside the bounds of an array is not possible because there are DET checks. This warning can be ignored
- RULE 11.5, DIR 1.1: 4-Byte-Alignment is ensured as SduDataPtr is aligned and we go i 8 forward.This cast allows easier (= safer) access to the data in the buffer Casting over void is necessary as otherwise some compilers complain
- RULE 11.5, DIR 1.1: Compiler ensures the correct alignment in the datastructure to which XcpResPtr points, thus casting is no problem. The cast allows easier (= safer) access to the data in the PDU. Casting over void is necessary as otherwise some compilers complain
- RULE 11.5, DIR 1.1: Compiler ensures the correct alignment in the datastructure, thus casting is no problem. The cast allows easier (= safer) access to

the data in the PDU. Casting over void is necessary as otherwise some compilers complain

- RULE 11.5, DIR 1.1: Compiler ensures the correct alignment in the datastructure, thus casting is no problem. The cast allows easier (= safer) access to the data in the PDU. Casting over void is necessary as otherwise some compilers complain
- RULE 11.5, DIR 1.1: Compiler ensures the correct alignment in the datastructure, thus casting is no problem.The cast allows easier (= safer) access to the data in the PDU. Casting over void is necessary as otherwise some compilers complain
- RULE 11.5, DIR 1.1: No problem with alignment as an OdtEntry is 1 byte
- RULE 11.5, DIR 1.1: Pointers are 32-bit-aligned so casting is no problem Casting over void is necessary as otherwise some compilers complain
- RULE 11.5, DIR 1.1: The calling layers ensure correct alignment. Casting over void is necessary as otherwise some compilers complain
- RULE 11.5, DIR 1.1: The calling layers ensure correct alignment. No other possibility to assign the pointer to the other pointer. Casting over void is necessary as otherwise some compilers complain
- RULE 11.5, DIR 1.1: The calling layers ensure correct alignment. This cast allows easier (= safer) access to the data in the buffer Casting over void is necessary as otherwise some compilers complain
- RULE 11.5, DIR 1.1: The calling layers ensure correct alignment. address is then used to do an atomic access to 32/64bit (machine width) Casting over void is necessary as otherwise some compilers complain
- RULE 11.5, DIR 1.1: The calling layers ensure correct alignment. address is then used to do an atomic access to 32/64bit (machine width) Casting over void is necessary as otherwise some compilers complain
- RULE 11.5, DIR 1.1: The calling layers ensure correct alignment; Cast to const is no problem, as the variable is only read DIR 1.1: The input parameters of rba_BswSrv_MemCopy() are declared as (void) for generic implementation of MemCopy.
- RULE 11.5, DIR 1.1: The pointer cast is required for copying the data from local structure to Daq pointer
- RULE 11.5, DIR 1.1: The pointer cast is required for copying the data from the Daq pointer to local structure
- RULE 11.5, DIR 1.1: This function is called with data (a PDU) from the CanIf/FrIf/EthIf which ensure 4-Byte-Alignment. The cast allows easier (= safer) access to the data in the PDU. Casting over void is necessary as otherwise some compilers complain
- RULE 11.5, DIR 1.1: This function is called with data (a PDU) from the CanIf/FrIf/EthIf which ensure 4-Byte-Alignment.The cast allows easier (= safer) access to the data in the PDU. Casting over void is necessary as otherwise some compilers complain

- RULE 11.6, DIR 1.1: The input parameters of rba_BswSrv_MemCopy() are declared as (void) for generic implementation of MemCopy.
- RULE 13.5: Conditions are evaluated from left to right. The function call is only done if the first condition is TRUE - this is intended behavior; Redesign not possible as while loop and not if - change would decrease maintainability and increase runtime
- RULE 13.5: Conditions are evaluated from left to right. The function call is only done if the first condition is TRUE - this is intended behavior; Redesign not possible as while loop instead of if possible - change would decrease maintainability and increase runtime
- RULE 13.5: Conditions are evaluated from left to right. The function call is only done if the first condition is TRUE - this is intended behavior; Redesign would decrease maintainability and increase runtime
- RULE 14.3: The condition will only be true in case of error. To have a simple DET-Reporting function this condition is encapsulated into a macro and cannot be left out
- RULE 15.4: more than one break is needed to avoid additional local breakloop variable
- RULE 2.2: Section 2.20.1 _TRANSPORT_LAYER_INDEX_OFFSET changes depending on the configuration thus the calculation is necessary if the value is not 0 (0 raises the MISRA violation)
- RULE 21.18, 18.1, DIR 4.1: Access outside the bounds of an array is not possible because there are DET checks. This warning can be ignored
- RULE 5.6: Type declaration is needed only for Xcp Resume Mode to simplify the copy procedure.
- RULE 5.6: Type declaration is needed only for Xcp on ETH. Declaration in any api will cause compiler errors for projects that use only Xcp on CAN.
- RULE 8.13: AddrPtrDst cannot be const as the data is modified in rba_D-me_Write
- RULE 8.13: AddrPtrDst cannot be const as this is only a template and thus not doing anything with the parameters.
- RULE 8.13: All transport layers have a functions with this signature. Only in XcpOnEth it is not used and thus empty. Pointer cannot be constant to keep signature identical.
- RULE 8.13: Content of XcpPacketPtr will be changed, when padding is enabled
- RULE 8.13: The object referenced by 'XcpBufferAligned' cannot be const as data is modified in rba_BswSrv_MemCopy
- RULE 8.13: The pointer AddrPtrDst cannot be const as data is modified in rba_BswSrv_MemCopy
- RULE 8.13: The pointer SrvPtr is copied from DataStream and it's contents are modified - therefore it cannot be const.

- RULE 8.13: The pointer XcpAddrPtr may be changed if XCP_STATIC_AD-DRESS_TRANSFORMATION is enabled - therefore it cannot be const.
- RULE 8.13: The pointer is for reference only and cannot be constant. Same function definition must be used for all transport layers.
- RULE 8.13: The pointers lastAddressPtr and lastValuePtr are changed in Xcp_CopyOdtEntries_Safe - therefore it cannot be const.
- RULE 8.13: The referenced object is modifiable and is passed to a non-const object, thus const is not used
- RULE 8.13: content of XcpResPacketPtr will be changed, when function is used

3 Contacting ETAS

3.1 Technical Support

Technical support is available to all users with a valid support contract. If you do not have a valid support contract, please contact your regional sales office (see below).

The best way to get technical support is by email. Any problems or questions about the use of the product should be sent to:

rta.hotline@etas.com

If you prefer to discuss your problem with the technical support team, you call the support hotline on:

+44 (0)1904 562624.

The hotline is available during normal office hours (0900-1730 GMT/BST).

3.2 ETAS HQ

ETAS GmbH Borsigstrasse 24 70469 Stuttgart Germany	Phone:	+49 711 3423-0
	Fax:	+49 711 3423-2106
	WWW:	www.etas.com

3.3 ETAS Local Sales and Support Offices

Contact details for your local sales office and local technical support team (where available) can be found on the ETAS web site:

ETAS subsidiaries	www.etas.com/en/contact.php
ETAS technical support	www.etas.com/en/hotlines.php